

MAKALAH
PROJECT ALGORITMA DAN PEMROGRAMAN

Makalah ini dibuat untuk memenuhi tugas mata kuliah algoritma dan pemrograman

Dosen Pengampu :

Lutfiah Maharani Siniwi, S.Stat., M.Stat.



Disusun oleh :

KELOMPOK B

ANGGOTA KELOMPOK :

Naufal Luqyana Athollah	(K1D025086)
Naila Putri Ardianti	(K1D025089)
Zulhilmy Muhammad Ash Shidqi	(K1D025095)
Muhammad Rafif Ardianto	(K1D025096)
Intan Malika Salsabilla	(K1D025105)

KEMENTRIAN PENDIDIKAN TINGGI, SAINS, DAN TEKNOLOGI
UNIVERSITAS JENDERAL SOEDIRMAN
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
PROGRAM STUDI STATISTIKA
PURWOKERTO

2026

DAFTAR ISI

DAFTAR ISI	i
BAB I.....	1
PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan	1
1.3 Ruang Lingkup	2
BAB II.....	3
ANALISIS MASALAH	3
2.1 Program 1	3
2.2 Program 2	3
2.3 Program 3	4
2.4 Program 4	5
2.5 Program 5	6
2.6 Program 6	7
2.7 Pembagian Tugas Kelompok	8
BAB III	11
PSEUDOCODE DAN FLOWCHART	11
3.1 Program 1	11
3.2 Program 2	12
3.3 Program 3	13
3.4 Program 4	15
3.5 Program 5	18
3.6 Program 6	20
BAB IV	23
IMPLEMENTASI PROGRAM.....	23

4.1 Program 1	23
4.2 Program 2	23
4.3 Program 3	24
4.4 Program 4	26
4.5 Program 5	29
4.6 Program 6	33
BAB V	36
PENJELASAN LINE-BY-LINE CODE	36
5.1 Program 1	36
5.2 Program 2	37
5.3 Program 3	38
5.4 Program 4	46
5.5 Program 5	48
5.6 Program 6	53
BAB VI	57
PENGUJIAN PROGRAM	57
6.1 Program 1	57
6.2 Program 2	62
6.3 Program 3	64
6.4 Program 4	69
6.5 Program 5	75
6.6 Program 6	79
BAB VII	85
PENUTUP	85
7.1 Kesimpulan	85
7.2 Kendala	85
7.3 Saran	85

LAMPIRAN87

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat menuntut mahasiswa untuk memiliki kemampuan berpikir logis, sistematis, dan mampu menyelesaikan berbagai permasalahan menggunakan komputer. Salah satu keterampilan dasar yang perlu dikuasai adalah kemampuan menyusun algoritma dan mengimplementasikannya ke dalam bahasa pemrograman.

Project akhir mata kuliah Algoritma dan Pemrograman dirancang untuk memberikan pengalaman praktis kepada mahasiswa dalam menerapkan konsep-konsep dasar pemrograman melalui berbagai studi kasus yang diberikan. Melalui project ini, mahasiswa tidak hanya memahami sintaks pemrograman, tetapi juga mampu menganalisis permasalahan, merancang algoritma, serta menjelaskan logika penyelesaian masalah yang digunakan.

Selain itu, project ini juga melatih kemampuan kerja sama tim, dokumentasi program, pengujian program, dan penggunaan alat kolaborasi dalam pengembangan perangkat lunak. Melalui proses pengerjaan project secara berkelompok, mahasiswa memperoleh pengalaman dalam bekerja secara kolaboratif serta mengkomunikasikan hasil implementasi program yang telah dibuat. Dengan demikian, project ini diharapkan dapat menjadi sarana bagi mahasiswa untuk mengintegrasikan pengetahuan teoritis dan keterampilan praktis yang telah diperoleh selama mengikuti mata kuliah Algoritma dan Pemrograman.

1.2 Tujuan

Tujuan pelaksanaan project ini adalah sebagai berikut:

1. Menganalisis permasalahan komputasional dengan menentukan input, proses, output, serta batasan masalah.
2. Menyusun algoritma dalam bentuk narasi, pseudocode, dan flowchart yang runtut.
3. Mengimplementasikan algoritma menggunakan dua bahasa pemrograman, yaitu R dan Python.
4. Membandingkan struktur logika program antara R dan Python pada kasus yang sama.
5. Menerapkan variabel, tipe data, operator, string, percabangan, fungsi, tuple/list/vector, dan operasi matematika.

6. Melakukan validasi input dan menangani kondisi khusus pada program.
7. Menjelaskan setiap baris kode secara logis, bukan sekadar menerjemahkan sintaks
8. Menguji program menggunakan beberapa skenario data dan mendokumentasikan hasilnya secara sistematis.
9. Bekerja sama dalam tim menggunakan repository GitHub dan membagi kontribusi secara adil.

1.3 Ruang Lingkup

Ruang lingkup project ini mencakup perancangan, implementasi, dan pengujian enam program yang diberikan pada tugas project akhir mata kuliah Algoritma dan Pemrograman, yaitu :

1. Program analisis teks untuk menghitung jumlah kata dan jumlah kalimat dalam suatu teks.
2. Program penggunaan string untuk menampilkan kalimat sesuai format yang ditentukan.
3. Program pencarian akar persamaan kuadrat yang dapat menentukan akar real maupun mengidentifikasi akar imajiner.
4. Program pengolahan NIP ASN untuk menampilkan informasi tanggal lahir berdasarkan delapan digit pertama NIP.
5. Program klasifikasi cluster tiga dimensi untuk menentukan keanggotaan suatu titik terhadap cluster A, B, atau C berdasarkan jarak terdekat.
6. Program perhitungan interval konfidensi proporsi berdasarkan nilai proporsi sampel, ukuran sampel, dan tingkat kepercayaan yang diberikan.

Dalam pelaksanaannya, setiap program akan melalui tahapan analisis masalah, penyusunan pseudocode, pembuatan flowchart, implementasi menggunakan Python dan R, penjelasan kode program, serta pengujian menggunakan beberapa skenario data. Hasil dari seluruh tahapan tersebut akan didokumentasikan dalam laporan project dan dipresentasikan.

BAB II

ANALISIS MASALAH

2.1 Program 1

1. Deskripsi program

Pada program 1, program ini digunakan untuk mencari jumlah kata dan jumlah kalimat dari sebuah teks yang dimasukkan pengguna. Teks yang diinput akan diproses dengan cara memisahkan kata berdasarkan spasi agar jumlah katanya dapat diketahui. Program juga menghitung banyaknya tanda titik (.). karena tanda tersebut dianggap sebagai akhir kalimat. Setelah semua proses selesai, hasil jumlah kata dan kalimat akan ditampilkan. Dengan program ini, pengguna bisa mengetahui isi teks secara lebih mudah dan sederhana.

2. Tabel input-proses-output

NO	KOMPONEN	KETERANGAN
1	Input	Teks atau paragraf yang diberikan
2	Proses 1	program akan memisahkan teks berdasarkan dengan spasi pada setiap kata untuk menghitung jumlah kata.
3	Proses 2	Program akan menghitung jumlah titik untuk menentukan jumlah kalimat.
4	Output	Jumlah kata dan kalimat pada teks yang diberikan.

3. Batasan masalah

Pada program 1 terdapat satu batasan masalah yaitu, titik (.) digunakan hanya sebagai akhir dari suatu kalimat.

2.2 Program 2

1. Deskripsi program

Pada program 2, program ini dipakai untuk membuat beberapa data string bernama K1, K2, K3, dan K4 sesuai dengan kalimat yang sudah disiapkan. Setiap kalimat dimasukkan ke variabel dengan penulisan tanda kutip yang benar supaya program bisa dijalankan tanpa error. Ada juga penggunaan escape character untuk menampilkan tanda kutip di dalam teks string. Setelah semua variabel selesai dibuat, program akan menampilkan isi variabel tersebut ke layar. Dari program ini, pengguna bisa belajar cara

membuat string dan memakai tanda kutip pada bahasa pemrograman Python maupun R.

2. Tabel Input-Proses-Output

NO	KOMPONEN	KETERANGAN
1	Input	Kalimat atau teks yang dimasukkan ke dalam variabel string K1 sampai K4.
2	Proses 1	Program menyimpan setiap kalimat ke dalam variabel string.
3	Proses 2	Program menggunakan tanda kutip dan escape character agar teks dapat dibaca dengan benar.
4	Output	Tampilan kalimat dari variabel K1, K2, K3, dan K4 sesuai ketentuan soal .

3. Batasan masalah program

Pada program 2 terdapat satu batasan masalah yaitu, kalimat yang digunakan pada program sudah ditentukan di dalam coding dan tidak diinput oleh pengguna saat program dijalankan.

2.3 Program 3

1. Deskripsi program

Pada program 3, program ini akan digunakan untuk mencari apakah suatu persamaan kuadrat mempunyai nilai akar-akar real dan menampilkan besar nilai akar-akar realnya. Data yang diperlukan untuk menjalankan program ini yaitu nilai a,b, dan c yang merupakan koefisien yang berupa bilangan real. Nilai koefisien akan diolah dengan mencari nilai diskriminan menggunakan rumus $D = b^2 - 4 * a * c$ setelah itu melakukan perhitungan untuk menghitung nilai akar-akar real. Dengan adanya program ini, pengguna dapat lebih mudah untuk mengetahui apakah suatu persamaan kuadrat mempunyai nilai akar-akar real.

2. Tabel Input-Proses-Output

NO	KOMPONEN	KETERANGAN
1	Input	Memasukkan nilai untuk variable a, b, dan c
2	Proses 1	Memeriksa apakah nilai a = 0
3	Proses 2	Menghitung diskriminan $D = b^2 - 4 * a * c$
4	Proses 3	Memeriksa (D>0, D=0, atau D<0).
5	Proses 4	Menghitung akar-akar real, ketika diskriminan > 0

6	Output	Menampilkan nilai akar-akar real
---	--------	----------------------------------

3. Batasan masalah

Pada program 3 terdapat beberapa batasan masalah untuk program tersebut sebagai berikut :

- Nilai a tidak boleh = 0 karena saat nilai a = 0 persamaan bukan merupakan persamaan kuadrat.
- Koefisien a,b, dan c berupa bilangan real.
- Program tidak dapat menampilkan bentuk bilangan untuk akar imajiner.

2.4 Program 4

1. Deskripsi program

Program ini dirancang untuk mengekstrak dan menampilkan informasi tanggal lahir seorang Aparatur Sipil Negara (ASN) berdasarkan Nomor Induk Pegawai (NIP) yang dimilikinya. NIP ASN terdiri dari 18 digit angka yang mengandung informasi identitas pegawai secara terstruktur dan terencode di dalamnya.

2. Tabel Input-Proses-Output

NO	KOMPONEN	KETERANGAN
1	INPUT	NIP ASN berupa string 18 digit
2	PROSES 1	Validasi panjang & format NIP
3	PROSES 2	Ambil digit 1-4 sebagai tahun
4	PROSES 3	Ambil digit 5-6 sebagai bulan
5	PROSES 4	Ambil digit 7–8 sebagai tanggal lahir
6	PROSES 5	Validasi rentang bulan (01-12)
7	PROSES 6	Tentukan maks hari sesuai bulan
8	PROSES 7	Validasi rentang tanggal

9	PROSES 8	Konversi angka bulan ke nama bulan via percabangan
10	OUTPUT	Tanggal lahir lengkap atau Error yang sesuai

3. Batasan masalah

Pada program 4 terdapat beberapa batasan masalah untuk program tersebut sebagai berikut :

- NIP yang dimasukkan diasumsikan selalu terdiri dari tepat 18 digit angka.
- Digit bulan bernilai antara 01–12
- Tanggal valid menyesuaikan jumlah hari tiap bulannya
- Program tidak melakukan validasi tahun lahir

2.5 Program 5

1. Deskripsi program

Pada program 5, program ini menerima sebuah titik sembarang dalam ruang 3 dimensi berbentuk tuple (x_1, x_2, x_3) . Program kemudian menghitung jarak euclid tersebut terhadap tiga pusat cluster A, B, dan C yang telah ditentukan. Cluster yang memiliki jarak paling kecil menjadi hasil klasifikasi titik tersebut dan jika terdapat jarak yang sama, maka tidak termasuk ke dalam salah satu cluster.

2. Tabel Input-Proses-Output

NO	KOMPONEN	KETERANGAN
1.	Input	koordinat x_1, x_2, x_3 berupa bilangan real yang dimasukkan satu per satu
2.	Proses 1	Mendeklarasikan koordinat pusat cluster: A(2,1,3), B(1,-4,6), C(-2,3,-2)
3.	Proses 2	Menghitung jarak Euclidean dari titik U terhadap cluster A, B, dan C menggunakan rumus $\sqrt{(x_1-y_1)^2 + (x_2-y_2)^2 + (x_3-y_3)^2}$
4.	Proses 3	Membandingkan ketiga jarak menggunakan percabangan untuk menentukan jarak yang lebih dekat/terkecil

	Output	menampilkan nilai jarak ke tiap cluster terdekat (A/B/C), serta hasil klasifikasi berupa cluster A, B, C, atau tidak dapat ditentukan jika da jarak yang sama
--	--------	---

3. Batasan masalah

Pada program 5 terdapat beberapa batasan masalah untuk program tersebut sebagai berikut :

- Input x1, x2, x3 harus berupa bilangan (bukan huruf atau karakter lain).
- Koordinat boleh bernilai negatif atau nol.
- Jika dua cluster memiliki jarak yang sama, program mendefinisikan tidak termasuk ke dalam salah satu cluster.

Kondisi khusus:

- Titik memiliki jarak yang sama persis kedua atau tiga pusat cluster
program tidak memaksakan memilih salah satu, melainkan menampilkan "tidak dapat ditentukan", karena secara matematis titik tersebut berada tepat di perbatasan/bidang pemisah antar cluster.
- Titik U tepat berada di pusat salah satu cluster (jarak = 0)
Program tetap menampilkan jarak 0.0000 dan otomatis menggolongkan titik tersebut ke cluster yang bersangkutan, karena jaraknya pasti paling kecil dibanding dua cluster lain.
- Koordinat input berupa bilangan negatif atau desimal
Rumus jarak Euclidean tetap berlaku untuk semua bilangan real, sehingga program tidak memerlukan validasi tambahan untuk tanda atau jenis bilangan.

2.6 Program 6

1. Deskripsi program

Pada program 6, program ini digunakan untuk menghitung interval konfidensi proporsi berdasarkan data sampel yang diberikan. Data yang diperlukan untuk menjalankan program ini yaitu nilai proporsi sampel (p_{hat}), ukuran sampel (n), dan tingkat signifikansi (α). Nilai-nilai tersebut akan diolah dengan menentukan nilai z sesuai tingkat signifikansi yang dipilih, kemudian menghitung margin of error menggunakan rumus interval konfidensi proporsi. Selanjutnya, program akan menghitung batas bawah dan batas atas interval konfidensi berdasarkan nilai margin of

error yang diperoleh. Dengan adanya program ini, pengguna dapat lebih mudah menentukan rentang estimasi proporsi populasi berdasarkan data sampel yang dimiliki.

2. Tabel Input-Proses-Output

NO	KOMPONEN	KETERANGAN
1	Input	Menginput nilai p_{hat} (proporsi sampel), n (ukuran sampel), dan α (tingkat signifikansi yang bernilai 0,05 atau 0,10).
2	Proses 1	Program melakukan validasi terhadap nilai input, yaitu memastikan p_{hat} berada pada rentang 0 sampai 1, n lebih besar dari 0, dan α bernilai 0,05 atau 0,10.
3	Proses 2	Jika input valid, program menentukan nilai z berdasarkan nilai α yang dipilih.
4	Proses 3	Program menghitung margin of error menggunakan rumus interval konfidensi proporsi.
5	Proses 4	Program menghitung batas bawah dan batas atas interval konfidensi proporsi berdasarkan nilai proporsi sampel dan margin of error yang diperoleh.
6	Output	Menampilkan batas bawah interval konfidensi, batas atas interval konfidensi, serta interval konfidensi dalam bentuk pasangan (batas bawah, batas atas).

3. Batasan masalah

Pada program 6 terdapat beberapa batasan masalah untuk program tersebut sebagai berikut:

- Nilai proporsi sampel (p_{hat}) harus berada pada rentang 0 sampai 1.
- Ukuran sampel (n) harus lebih besar dari 0.
- Nilai α yang diperbolehkan hanya 0,05 dan 0,10.
- Program menggunakan pendekatan distribusi normal dengan nilai z yang sudah ditentukan untuk masing-masing α .

Program hanya menghitung interval konfidensi untuk satu nilai proporsi sampel dalam satu kali eksekusi.

2.7 Pembagian Tugas Kelompok

Agar penyelesaian project lebih efisien, maka dilakukan pembagian tugas untuk mengerjakan project yang akan ditampilkan dalam tabel sebagai berikut :

NO	NAMA	BAGIAN	TUGAS
1	NAUFAL LUQYANA ATHOLLAH	PROGRAM 6	Membuat analisis masalah, tabel input-proses-output, menentukan batasan masalah program, membuat psedocode program, membuat flowchart program, menuliskan algoritma R dan python, membuat tabel penjelasan setiap baris kode, dan membuat tabel pengujian.
2	NAILA PUTRI ARDIANTI	PROGRAM 3	Membuat analisis masalah, tabel input-proses-output, menentukan batasan masalah program, membuat psedocode program, membuat flowchart program, menuliskan algoritma R dan python, membuat tabel penjelasan setiap baris kode, dan membuat tabel pengujian.
3	ZHULHIMLY MUHAMMAD ASH SHIDQI	PROGRAM 5	Membuat analisis masalah, tabel input-proses-output, menentukan batasan masalah program, membuat psedocode program, membuat flowchart program, menuliskan algoritma R dan python, membuat tabel penjelasan setiap baris kode, dan membuat tabel pengujian.
4	MUHAMMAD RAFIF ARDIANTO	PROGRAM 4	Membuat analisis masalah, tabel input-proses-output, menentukan batasan masalah program, membuat psedocode program, membuat flowchart program, menuliskan algoritma R dan python, membuat tabel penjelasan setiap baris kode, dan membuat tabel pengujian.
5	INTAN MALIKA SALSABILLA	PROGRAM 1 & 2	Membuat analisis masalah, tabel input-proses-output, menentukan batasan masalah program, membuat psedocode program, membuat flowchart program, menuliskan algoritma R dan python, membuat tabel penjelasan

			setiap baris kode, dan membuat tabel pengujian.
--	--	--	--

BAB III

PSEUDOCODE DAN FLOWCHART

3.1 Program 1

1. Pseudocode

START

DEKLARASI

teks : string

jumlah_kata : integer

jumlah_kalimat : integer

DESKRIPSI

Input teks

jumlah_kata $\leftarrow$ hitung jumlah kata dari teks

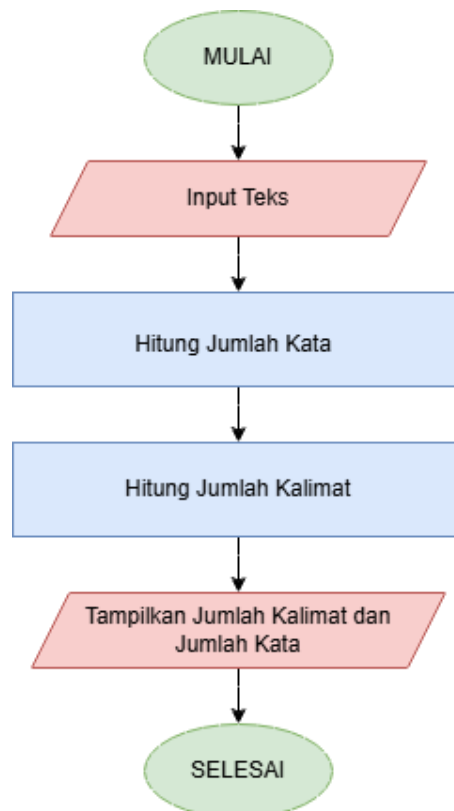
jumlah_kalimat $\leftarrow$ hitung jumlah tanda titik (.)

Tampilkan jumlah_kalimat

Tampilkan jumlah_kata

END

2. Flowchart



3.2 Program 2

1. Pseudocode

START

DEKLARASI :

K1, K2, K3, K4 : string

DESKRIPSI :

Masukkan kalimat ke variabel K1

Masukkan kalimat ke variabel K2

Masukkan kalimat ke variabel K3

Masukkan kalimat ke variabel K3

Tampilkan K1

Tampilkan K2

Tampilkan K3

Tampilkan K4

END

2. Flowchart



3.3 Program 3

1. Pseudocode

START

DEKLARASI:

a, b, c : integer

Diskriminan : integer

x, x1, x2 : integer

DESKRIPSI:

Input a, b, c

IF a == 0 THEN

Tampilkan "bukan persamaan kuadrat"

ELSE

Diskriminan $\leftarrow (b * b) - (4 * a * c)$

IF Diskriminan > 0 THEN

$x1 \leftarrow (-b + \text{Sqrt}(\text{Diskriminan})) / (2 * a)$

$x2 \leftarrow (-b - \text{Sqrt}(\text{Diskriminan})) / (2 * a)$

Tampilkan "Persamaan memiliki akar-akar real:"

Tampilkan "x1 = ", format(x1, 3 desimal)

Tampilkan "x2 = ", format(x2, 3 desimal)

ELSE IF Diskriminan == 0 THEN

$x1 \leftarrow -b / (2 * a)$

$x2 \leftarrow -b / (2 * a)$

Tampilkan "Persamaan memiliki akar kembar:"

Tampilkan "x1 = ", format(x1, 3 desimal)

Tampilkan "x2 = ", format(x2, 3 desimal)

ELSE

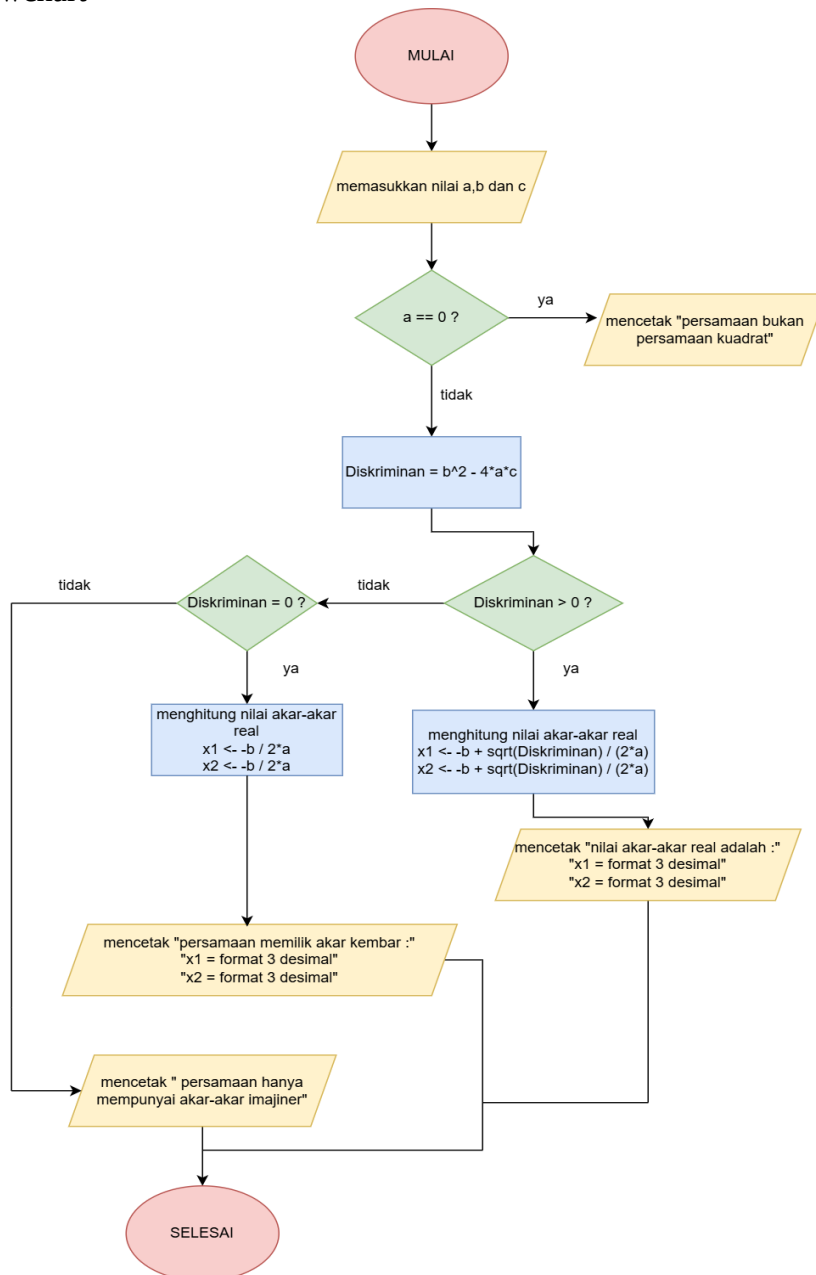
Tampilkan "Persamaan hanya memiliki akar-akar imajiner"

ENDIF

ENDIF

END

2. Flowchart



3.4 Program 4

1. Pseudocode

START

DEKLARASI :

nip : string

tahun: string

bulan: integer

tanggal: integer

maks_hari: integer

nama_bulan: string

DESKRIPSI :

INPUT nip

IF panjang(nip) $\neq$ 18 ATAU nip bukan semua digit THEN

OUTPUT "Error: NIP harus terdiri dari tepat 18 digit angka."

STOP

END IF

tahun $\leftarrow$ nip[0:4]

bulan $\leftarrow$ integer(nip[4:6])

tanggal $\leftarrow$ integer(nip[6:8])

IF bulan < 1 ATAU bulan > 12 THEN

OUTPUT "Error: Bulan pada NIP tidak valid (harus 01–12)."

STOP

END IF

IF bulan $\in$ {1,3,5,7,8,10,12} THEN

maks_hari $\leftarrow$ 31

ELSE IF bulan $\in$ {4,6,9,11} THEN

maks_hari $\leftarrow$ 30

ELSE IF bulan = 2 THEN

maks_hari $\leftarrow$ 29

END IF

IF tanggal < 1 ATAU tanggal > maks_hari THEN

OUTPUT "Error: Tanggal tidak valid untuk bulan ini (harus 01–maks_hari)."

STOP

END IF

IF bulan = 1 THEN nama_bulan $\leftarrow$ "Januari"

ELSE IF bulan = 2 THEN nama_bulan $\leftarrow$ "Februari"

ELSE IF bulan = 3 THEN nama_bulan $\leftarrow$ "Maret"

ELSE IF bulan = 4 THEN nama_bulan $\leftarrow$ "April"

ELSE IF bulan = 5 THEN nama_bulan $\leftarrow$ "Mei"

ELSE IF bulan = 6 THEN nama_bulan $\leftarrow$ "Juni"

ELSE IF bulan = 7 THEN nama_bulan $\leftarrow$ "Juli"

ELSE IF bulan = 8 THEN nama_bulan $\leftarrow$ "Agustus"

ELSE IF bulan = 9 THEN nama_bulan $\leftarrow$ "September"

ELSE IF bulan = 10 THEN nama_bulan $\leftarrow$ "Oktober"

ELSE IF bulan = 11 THEN nama_bulan $\leftarrow$ "November"

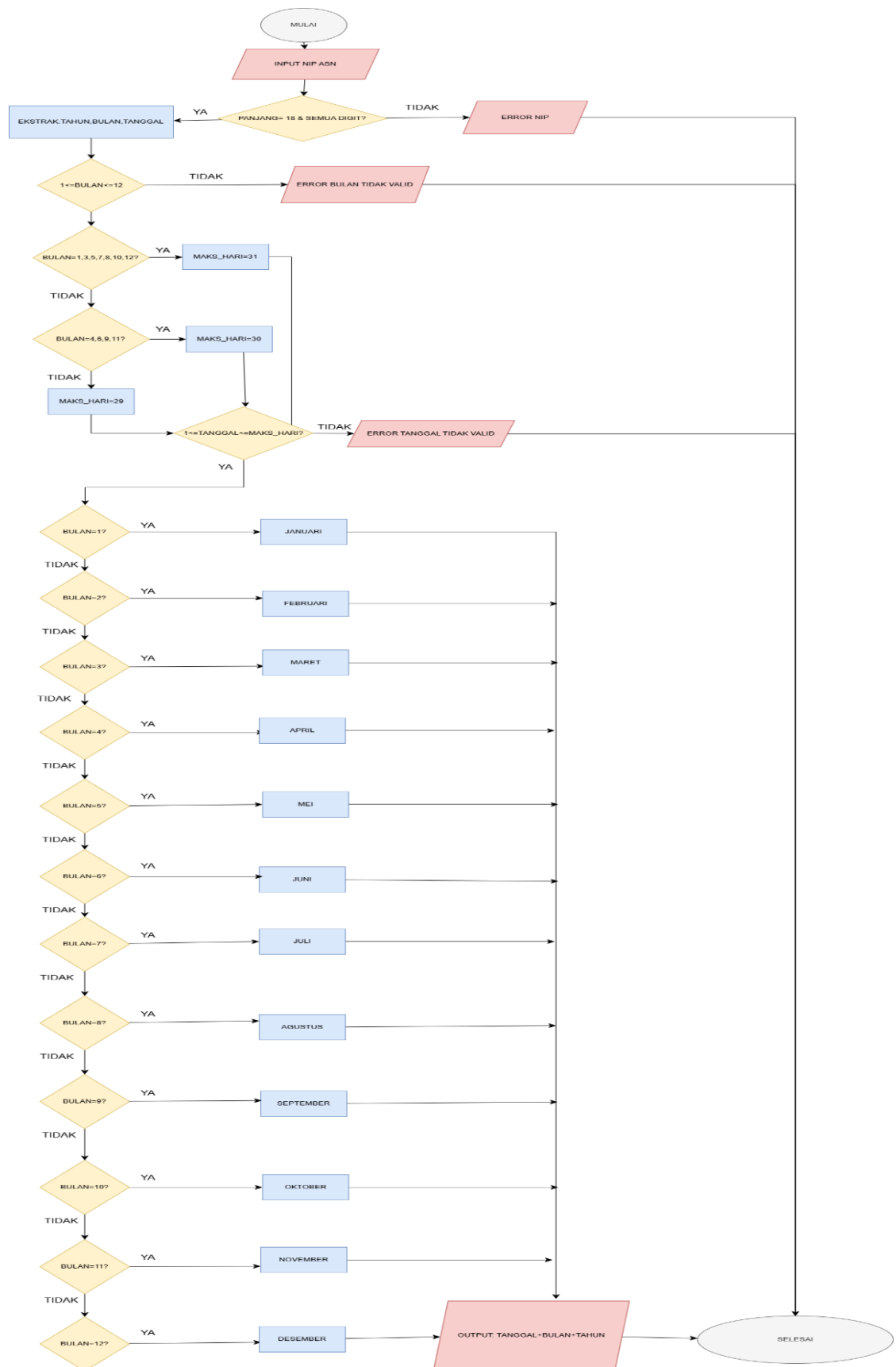
ELSE IF bulan = 12 THEN nama_bulan $\leftarrow$ "Desember"

END IF

OUTPUT "Tanggal lahir: " + tanggal + " " + nama_bulan + " " + tahun

END

2. Flowchart



3.5 Program 5

1. Pseudocode

START

Deklarasikan pusat cluster:

$A = (2,1,3)$

$B = (1,-4,6)$

$C = (-2,3,-2)$

Baca input titik $U(x_1, x_2, x_3)$

FUNCTION

hitung_jarak(U , cluster):

hitung selisih koordinat

kuadratkan setiap selisih

jumlahkan hasil kuadrat

akar kuadrat hasil penjumlahan

RETURN jarak

Hitung:

jarak_A = hitung_jarak(U , A)

jarak_B = hitung_jarak(U , B)

jarak_C = hitung_jarak(U , C)

IF jarak_A lebih dekat dari B dan C:

cluster = A

ELSE IF jarak_B lebih dekat dari A dan C:

cluster = B

ELSE IF jarak_C lebih dekat A dan B:

cluster = C

ELSE:

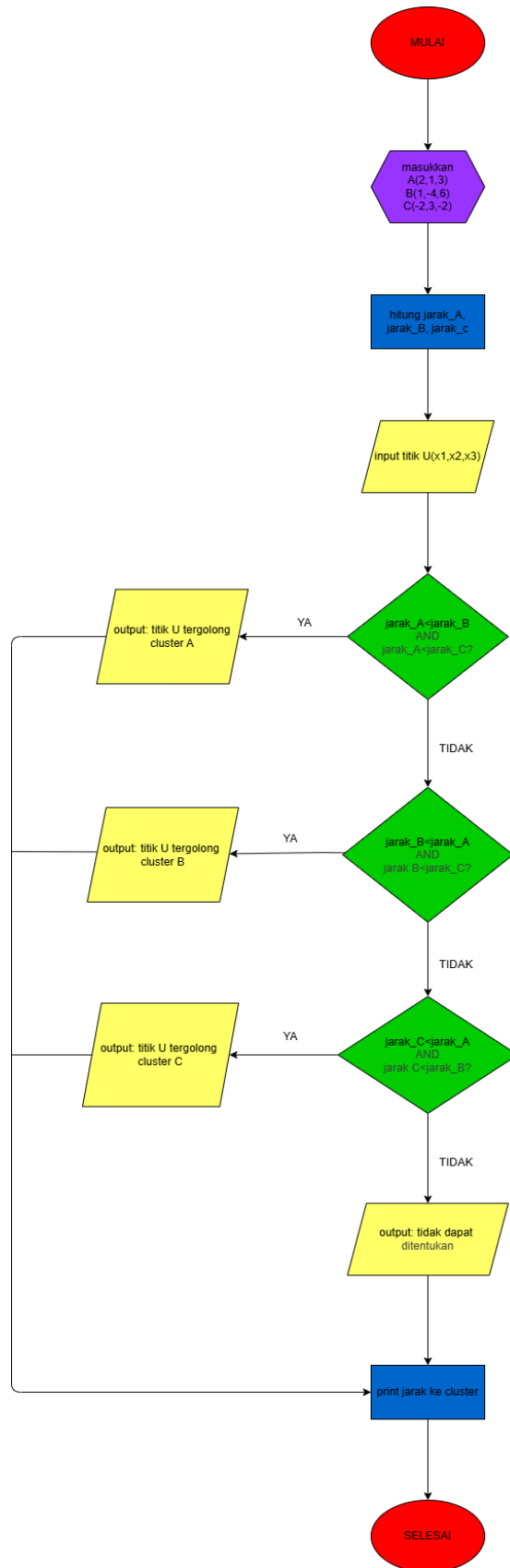
cluster = tidak dapat ditentukan

Tampilkan jarak tiap cluster

Tampilkan cluster hasil klasifikasi

END

2. Flowchart



3.6 Program 6

1. Pseudocode

START

DEKLARASI:

p_hat : Real
n : Integer
alpha : Real
z : Real
margin_error : Real
batas_bawah : Real
batas_atas : Real
hasil : Record {BB, BA}

DESKRIPSI:

Input p_hat

Input n

Input alpha

IF p_hat < 0 OR p_hat > 1 THEN

Tampilkan "Error: Proporsi sampel harus berada pada rentang 0 sampai 1"

ELSE IF n <= 0 THEN

Tampilkan "Error: Ukuran sampel harus lebih dari 0"

ELSE IF alpha ≠ 0.05 AND alpha ≠ 0.10 THEN

Tampilkan "Error: Alpha harus bernilai 0.05 atau 0.10"

ELSE

IF alpha = 0.05 THEN

z = 1.96

ELSE IF alpha = 0.10 THEN

z = 1.645

ENDIF

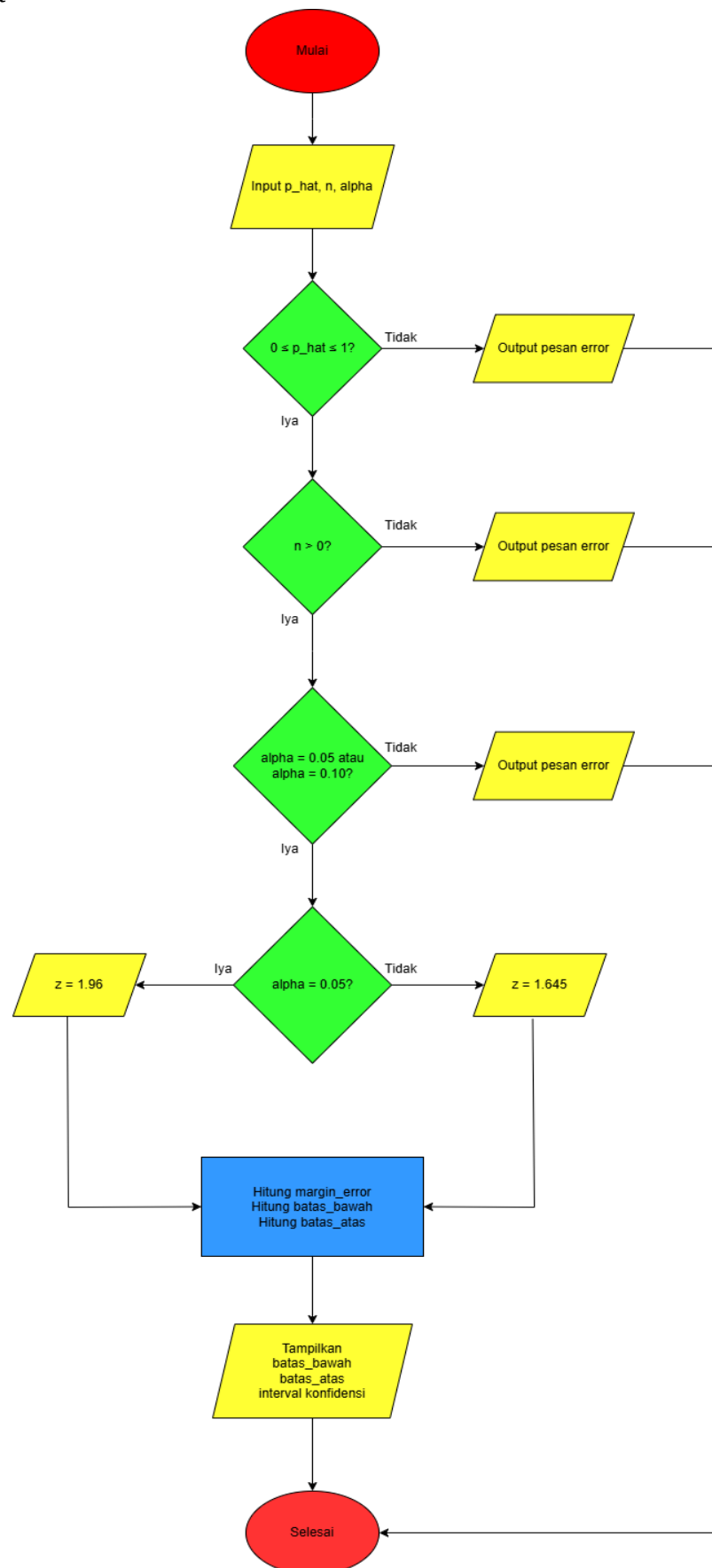
margin_error ← $z \times \sqrt{(p_hat \times (1 - p_hat)) / n}$

```
batas_bawah ← p_hat - margin_error
batas_atas ← p_hat + margin_error

hasil.BB ← batas_bawah
hasil.BA ← batas_atas

Tampilkan "Batas bawah = ", hasil.BB
Tampilkan "Batas atas = ", hasil.BA
Tampilkan "Interval konfidensi = (", hasil.BB, ", ", hasil.BA, ")"
ENDIF
END
```

2. Flowchart



BAB IV

IMPLEMENTASI PROGRAM

4.1 Program 1

1. Source R

```
# Menghitung jumlah kata dan jumlah kalimat pada soal nomor 1
teks <- "Masukkan Teks"
jumlah_kata <- length(strsplit(teks, " ")[[1]])
jumlah_kalimat <- lengths(regmatches(teks, gregexpr("\\.", teks)))

cat("Teks tersebut memuat",
    jumlah_kalimat,
    "kalimat dan",
    jumlah_kata,
    "kata.")
```

2. Source Python

```
# Menghitung jumlah kalimat dan kata
teks = "Masukkan Teks"
jumlah_kata = len(teks.split())
jumlah_kalimat = teks.count(".")
print("Teks tersebut memuat", jumlah_kalimat,
    "kalimat dan", jumlah_kata, "kata.")
```

4.2 Program 2

1. Source R

```
#bentuk umum
K1 = "masukkan teks."
K2 = 'Masukkan teks, "Masukkan teks."'
K3 = "\"Masukkan teks\""
K4 = "Masukkan teks."

cat(K1, "\n")
cat(K2, "\n")
cat(K3, "\n")
```

```
cat(K4, "\n")
```

2. Source Python

```
#bentuk umum
K1 = "masukkan teks."
K2 = 'Masukkan teks, "Masukkan teks."'
K3 = "\"Masukkan teks\""
K4 = "Masukkan teks."

print(K1)
print(K2)
print(K3)
print(K4)
```

4.3 Program 3

1. Source R

```
#mencari akar-akar real persamaan kuadrat
#mendefinisikan variabel
a <- as.numeric(readline(prompt = "masukkan nilai a: "))
b <- as.numeric(readline(prompt = "masukkan nilai b: "))
c <- as.numeric(readline(prompt = "masukkan nilai c: "))

#mengidentifikasi persamaan kuadrat
if (a == 0) {
  cat("persamaan bukan persamaan kuadrat\n")
} else {
  Diskriminan <- b^2 - 4*a*c
}

#mencari akar-akar persamaan
if (Diskriminan > 0) {
  x1 <- (-b + sqrt(Diskriminan)) / (2*a)
  x2 <- (-b - sqrt(Diskriminan)) / (2*a)
```

```

        cat(sprintf("akar-akar real adalah: x1 = %.3f dan x2 = %.3f\n", x1, x2))
    } else if (Diskriminan == 0) {
        x1 <- -b / (2*a)
        x2 <- -b / (2*a)
        cat(sprintf("persamaan memiliki akar kembar : x1 = %.3f dan x2 = %.3f\n", x1,
x2))
    } else {
        cat("persamaan hanya mempunyai akar-akar imajiner\n")
    }
}

```

2. Source Python

```

import math

#menghitung persamaan kuadrat
# Mendefinisikan nilai setiap variabel
a = float(input("Masukkan nilai a: "))
b = float(input("Masukkan nilai b: "))
c = float(input("Masukkan nilai c: "))

# identifikasi bentuk persamaan
if a == 0:
    print("bukan persamaan kuadrat")
else:
    Diskriminan = b**2 - 4*a*c

    # Pengecekan kondisi akar
    if Diskriminan > 0:
        x1 = (-b + math.sqrt(Diskriminan)) / (2*a)
        x2 = (-b - math.sqrt(Diskriminan)) / (2*a)

    # Menampilkan hasil dengan format 3 desimal
    print(f"\nPersamaan memiliki akar-akar real:")
    print(f"x1 = {x1:.3f}")
    print(f"x2 = {x2:.3f}")

```

```

elif Diskriminan == 0:
    x1 = -b / (2*a)
    x2 = -b / (2*a)
    print(f"\nPersamaan memiliki akar kembar:")
    print(f"x1 = {x1:.3f}")
    print(f"x2 = {x2:.3f}")
else:
    print("\nPersamaan hanya memiliki akar-akar imajiner")

```

4.4 Program 4

1. Source R

```

#program 4
nip <- readline("Masukkan NIP ASN: ")

if (nchar(nip) != 18 || grepl("[^0-9]", nip)) {
  cat("Error: NIP harus terdiri dari tepat 18 digit angka.\n")
} else {
  tahun <- substr(nip, 1, 4)
  bulan <- as.integer(substr(nip, 5, 6))
  tanggal <- as.integer(substr(nip, 7, 8))

  if (bulan < 1 || bulan > 12) {
    cat("Error: Bulan pada NIP tidak valid (harus 01–12).\n")
  } else {
    if (bulan %in% c(1, 3, 5, 7, 8, 10, 12)) {
      maks_hari <- 31
    } else if (bulan %in% c(4, 6, 9, 11)) {
      maks_hari <- 30
    } else if (bulan == 2) {
      maks_hari <- 29
    }

    if (tanggal < 1 || tanggal > maks_hari) {

```

```

cat(paste0("Error: Tanggal pada NIP tidak valid untuk bulan ini (harus 01–",
          sprintf("%02d", maks_hari), ").\n"))
} else {
  if (bulan == 1) {
    nama_bulan <- "Januari"
  } else if (bulan == 2) {
    nama_bulan <- "Februari"
  } else if (bulan == 3) {
    nama_bulan <- "Maret"
  } else if (bulan == 4) {
    nama_bulan <- "April"
  } else if (bulan == 5) {
    nama_bulan <- "Mei"
  } else if (bulan == 6) {
    nama_bulan <- "Juni"
  } else if (bulan == 7) {
    nama_bulan <- "Juli"
  } else if (bulan == 8) {
    nama_bulan <- "Agustus"
  } else if (bulan == 9) {
    nama_bulan <- "September"
  } else if (bulan == 10) {
    nama_bulan <- "Oktober"
  } else if (bulan == 11) {
    nama_bulan <- "November"
  } else if (bulan == 12) {
    nama_bulan <- "Desember"
  }

  cat(paste0("Tanggal lahir: ", tanggal, " ", nama_bulan, " ", tahun, "\n"))
}
}
}

```

2. Source Python

```

#program 4

nip = input("Masukkan NIP ASN: ")

if len(nip) != 18 or not nip.isdigit():
    print("Error: NIP harus terdiri dari tepat 18 digit angka.")
else:
    tahun = nip[0:4]
    bulan = int(nip[4:6])
    tanggal = int(nip[6:8])

    if bulan < 1 or bulan > 12:
        print("Error: Bulan pada NIP tidak valid (harus 01–12).")
    else:
        if bulan in [1, 3, 5, 7, 8, 10, 12]:
            maks_hari = 31
        elif bulan in [4, 6, 9, 11]:
            maks_hari = 30
        elif bulan == 2:
            maks_hari = 29

        if tanggal < 1 or tanggal > maks_hari:
            print(f"Error: Tanggal pada NIP tidak valid untuk bulan ini (harus 01–{maks_hari:02d}).")
        else:
            if bulan == 1:
                nama_bulan = "Januari"
            elif bulan == 2:
                nama_bulan = "Februari"
            elif bulan == 3:
                nama_bulan = "Maret"
            elif bulan == 4:
                nama_bulan = "April"
            elif bulan == 5:

```

```

        nama_bulan = "Mei"
    elif bulan == 6:
        nama_bulan = "Juni"
    elif bulan == 7:
        nama_bulan = "Juli"
    elif bulan == 8:
        nama_bulan = "Agustus"
    elif bulan == 9:
        nama_bulan = "September"
    elif bulan == 10:
        nama_bulan = "Oktober"
    elif bulan == 11:
        nama_bulan = "November"
    elif bulan == 12:
        nama_bulan = "Desember"

    print(f"Tanggal lahir: {tanggal} {nama_bulan} {tahun}")

```

4.5 Program 5

1. Source R

```

#Program Klasifikasi Cluster 3 Dimensi
#FUNGSI: menghitung jarak Euclidean antara dua titik 3 Dimensi
hitung_jarak <- function(titik_U, pusat_cluster) {
    x1<-titik_U[1];y1<-pusat_cluster[1]
    x2<-titik_U[2];y2<-pusat_cluster[2]
    x3<-titik_U[3];y3<-pusat_cluster[3]

    jarak<-sqrt(
        (x1-y1)^2+
        (x2-y2)^2+
        (x3-y3)^2
    )
    return(jarak)
}

```

```

}

#Fungsi menentukan cluster dari titik U
klasifikasi_cluster<-function(titik_U){
#Konstanta pusat masing-masing cluster
pusat_A<-c(2,1,3)
pusat_B<-c(1,-4,6)
pusat_C<-c(-2,3,-2)

#Hitung jarak ke tiap pusat cluster
jarak_A<-hitung_jarak(titik_U,pusat_A)
jarak_B<-hitung_jarak(titik_U,pusat_B)
jarak_C<-hitung_jarak(titik_U,pusat_C)

#Tentukan cluster terdekat
if(jarak_A<jarak_B&&jarak_A<jarak_C){
  cluster<-"A"
}else if(jarak_B<jarak_A&&jarak_B<jarak_C){
  cluster<-"B"
}else if(jarak_C < jarak_A && jarak_C < jarak_B){
  cluster<-"C"
}else{
  cluster<-"tidak dapat ditentukan"
}

#Kembalikan hasil sebagai list
return(list(
  cluster=cluster,
  jarak_A=jarak_A,
  jarak_B=jarak_B,
  jarak_C=jarak_C
))
}

```

```

#Input angka
x1<-as.numeric(readline("Masukkan x1: "))
x2<-as.numeric(readline("Masukkan x2: "))
x3<-as.numeric(readline("Masukkan x3: "))

#Simpan sebagai vector 3 elemen
titik_U<-c(x1,x2,x3)

#Panggil fungsi klasifikasi
hasil<-klasifikasi_cluster(titik_U)

#Output hasil
cat(sprintf("\nTitik U(%.4g,%.4g,%.4g)\n",x1,x2,x3))
cat(sprintf("Jarak ke Cluster A:%.4f\n",hasil$jarak_A))
cat(sprintf("Jarak ke Cluster B:%.4f\n",hasil$jarak_B))
cat(sprintf("Jarak ke Cluster C:%.4f\n",hasil$jarak_C))
cat(sprintf("\nTitik U tergolong Cluster %s\n",hasil$cluster))

```

2. Source Python

```

#Program Klasifikasi Cluster 3 Dimensi
import math

#Fungsi menghitung jarak Euclidean antara dua titik 3D
def hitung_jarak(titik_U,pusat_cluster):
    x1,x2,x3= titik_U      #koordinat titik U
    y1,y2,y3=pusat_cluster  #koordinat pusat cluster
    jarak=math.sqrt(
        (x1-y1)**2 +
        (x2-y2)**2 +
        (x3-y3)**2
    )
    return jarak

#Fungsi menentukan cluster dari titik U

```

```

def klasifikasi_cluster(titik_U):
    #Konstanta pusat masing-masing cluster
    pusat_A=(2,1,3)
    pusat_B=(1,-4,6)
    pusat_C=(-2,3,-2)

    #Hitung jarak ke tiap pusat cluster
    jarak_A=hitung_jarak(titik_U,pusat_A)
    jarak_B=hitung_jarak(titik_U,pusat_B)
    jarak_C=hitung_jarak(titik_U,pusat_C)

    #Tentukan cluster terdekat
    if jarak_A<jarak_B and jarak_A<jarak_C:
        cluster="A"
    elif jarak_B<jarak_A and jarak_B<jarak_C:
        cluster="B"
    elif jarak_C<jarak_A and jarak_C<jarak_B:
        cluster="C"
    else:
        cluster="Tidak dapat ditentukan"

    return cluster,jarak_A,jarak_B,jarak_C

#Input dari pengguna
x1=float(input("Masukkan x1:"))
x2=float(input("Masukkan x2:"))
x3=float(input("Masukkan x3:"))

#Simpan sebagai tuple 3 elemen
titik_U=(x1,x2,x3)

#Panggil fungsi klasifikasi
cluster,jarak_A,jarak_B,jarak_C=klasifikasi_cluster(titik_U)

```

```
#Output hasil
print(f"Titik U({x1}, {x2}, {x3})")
print(f"Jarak ke Cluster A:{jarak_A:.4f}")
print(f"Jarak ke Cluster B:{jarak_B:.4f}")
print(f"Jarak ke Cluster C:{jarak_C:.4f}")
print(f"Titik U tergolong Cluster {cluster}")
```

4.6 Program 6

1. Source R

```
# Fungsi
hitung_interval <- function(p_hat, n, alpha) {

  if (alpha == 0.05) {
    z <- 1.96
  } else if (alpha == 0.10) {
    z <- 1.645
  }

  margin_error <- z * sqrt((p_hat * (1 - p_hat)) / n)

  batas_bawah <- p_hat - margin_error
  batas_atas <- p_hat + margin_error

  return(list(
    BB = batas_bawah,
    BA = batas_atas
  )
)
}

# Input
p_hat <- ("Input nilai p_hat")
```

```

n <- ("Input nilai n")
alpha <- ("Input nilai alpha")

# Validasi
if (p_hat < 0 || p_hat > 1) {
  cat("Error: p_hat harus berada antara 0 dan 1\n")
} else if (n <= 0) {
  cat("Error: ukuran sampel harus lebih dari 0\n")
} else if (alpha != 0.05 && alpha != 0.10) {
  cat("Error: alpha harus 0.05 atau 0.10\n")
} else {
  hasil <- hitung_interval(p_hat, n, alpha)

  #Menampilkan Hasil
  cat("Batas bawah =", hasil$BB, "\n")
  cat("Batas atas =", hasil$BA, "\n")
  cat("Interval konfidensi = (", round(hasil$BB, 4), ", ", round(hasil$BA, 4),
    ")\n")
}

```

2. Source Python

```

#KONDISI UMUM
import math

# Fungsi
def hitung_interval(p_hat, n, alpha):

    if alpha == 0.05:
        z = 1.96
    elif alpha == 0.10:
        z = 1.645

    margin_error = z * math.sqrt((p_hat * (1 - p_hat)) / n)

```

```

    batas_bawah = p_hat - margin_error
    batas_atas = p_hat + margin_error

    return {
        "BB": batas_bawah,
        "BA": batas_atas
    }

# Input
p_hat = ("Input nilai p_hat")
n = ("Input nilai n")
alpha = ("Input nilai alpha")

# Validasi
if p_hat < 0 or p_hat > 1:
    print("Error: Proporsi sampel harus berada pada rentang 0 sampai 1")

elif n <= 0:
    print("Error: Ukuran sampel harus lebih dari 0")

elif alpha != 0.05 and alpha != 0.10:
    print("Error: Alpha harus bernilai 0.05 atau 0.10")

else:
    hasil = hitung_interval(p_hat, n, alpha)

# Menampilkan Hasil
print("Batas Bawah =", hasil["BB"])
print("Batas Atas =", hasil["BA"])
print("Interval konfidensi = (", round(hasil["BB"], 4), ",", round(hasil["BA"], 4),
    ")")

```

BAB V

PENJELASAN LINE-BY-LINE CODE

5.1 Program 1

Berikut adalah penjelasan line-by-line code program 1 :

NO	BARIS CODE PYTHON	PENJELASAN
1	teks = "..."	Membaca teks yang di input pengguna lalu menyimpannya ke dalam variabel teks untuk dapat di proses lebih lanjut.
2	jumlah_kata = len(teks.split())	Memisahkan teks menjadi beberapa kata berdasarkan spasi menggunakan split (), kemudian menghitung jumlah kata menggunakan fungsi len().
3	jumlah_kalimat = teks.count(".")	Menghitung jumlah tanda titik (.) untuk menentukan jumlah kalimat pada teks menggunakan fungsi count(). pada setiap teks tanda titik (.) dianggap sebagai akhir kalimat.
4	print("Teks tersebut memuat", jumlah_kalimat, "kalimat dan", jumlah_kata, "kata.")	Untuk menampilkan hasil akhir yaitu jumlah kata dan jumlah kalimat pada teks.

NO	BARIS CODE R	PENJELASAN
1	teks <- "..."	Menyimpan paragraf teks ke dalam variabel teks agar dapat diproses lebih lanjut.
2	jumlah_kata <- length(strsplit(teks, " ")[1])	Memisahkan teks berdasarkan spasi menggunakan strsplit(), lalu menghitung banyak kata menggunakan length().

3	jumlah_kalimat <- lengths(regmatches(teks, gregexpr("\\.", teks)))	Menghitung jumlah tanda titik (.) untuk menentukan jumlah kalimat pada teks. pada setiap teks tanda titik (.) dianggap sebagai akhir kalimat.
4	cat("Teks tersebut memuat",	Untuk menampilkan hasil akhir.
5	jumlah_kalimat,	Menampilkan jumlah kalimat yang telah dihitung sebelumnya
6	"kalimat dan",	teks tambahan agar hasil lebih mudah untuk dipahami
7	jumlah_kata,	menampilkan jumlah kata hasil perhitungan program.
8	"kata.")	teks tambahan sekaligus mengakhiri output.

5.2 Program 2

Berikut adalah penjelasan line-by-line code program 2 :

NO	BARIS KODE PHYTON	PENJELASAN
1	K1 = "Masukkan Teks"	Membuat variabel K1 yang berisi kalimat pertama yang bertipe string.
2	K2 = "Masukkan Teks"	Membuat variabel K2 dengan tanda kutip ganda di dalam string.
3	K3 = "Masukkan Teks"	Membuat variabel K3 menggunakan escape character \" agar tanda kutip dalam ditampilkan.
4	K4 = "Masukkan Teks"	Membuat variabel K4 yang berisi teks dan kode tertentu.

5	Print (K1)	Menampilkan isi variabel K1.
6	Print (K2)	Menampilkan isi variabel K2.
7	Print (K3)	Menampilkan isi variabel K3.
8	Print (K4)	Menampilkan isi variabel K4.

NO	BARIS KODE R	PENJELASAN
1	K1 = "Masukkan Teks"	Membuat variabel K1 yang berisi kalimat pertama yang bertipe string.
2	K2 = "Masukkan Teks"	Membuat variabel K2 yang berisi kalimat dengan tanda kutip ganda di dalam string.
3	K3 = "Masukkan Teks"	Membuat variabel K3 menggunakan escape character \" agar tanda kutip dalam ditampilkan.
4	K4 = "Masukkan Teks"	Membuat variabel K4 yang berisi teks dan kode tertentu.
5	cat (K1, "\n")	Menampilkan isi variabel K1 lalu pindah ke baris baru menggunakan \n.
6	cat (K2, "\n")	Menampilkan isi variabel K2 lalu pindah ke baris baru menggunakan \n.
7	cat (K3, "\n")	Menampilkan isi variabel K3 lalu pindah ke baris baru menggunakan \n.
8	cat (K4, "\n")	Menampilkan isi variabel K1 lalu pindah ke baris baru menggunakan \n.

5.3 Program 3

Berikut adalah penjelasan line-by-line code program 3 :

NO	BARIS KODE PHYTON	PENJELASAN
1	Import math	Memanggil modul bawaan yang berfungsi agar pengguna dapat menggunakan fungsi matematika tingkat lanjut. Cara untuk mengatifikannya, setelah mengetik kode langsung jalankan dan python akan masuk ke mode math
2	#menghitung persamaan kuadrat	Sebagai penanda proses yang sedang dilakukan oleh programmer, tidak akan mempengaruhi jalannya program
3	a = float (input("masukkan nilai a: "))	Pada bagian input program akan membaca nilai a yang dituliskan programmer kemudian akan disimpan ke variabel a, sebelum itu bentuknya diubah menjadi float agar program menjadi lebih fleksibel karena nilai koefisien tidak selalu bernilai bilangan bulat, jadi program tetap bisa melakukan eksekusi untuk bilangan bulat maupun desimal baru setelah itu menyimpannya ke variabel a. Konsep untuk variabel b dan c juga sama. Ketiga data ini digunakan untuk menentukan nilai diskriminan dan juga pencarian nilai akar-akar real.
4	#identifikasi bentuk persamaan	Sebagai penanda untuk memasuki proses pemeriksaan persamaan Kudarat. Kode ini tidak akan mempengaruhi jalannya program.
5	If a == 0:	Kode if digunakan untuk melakukan pemeriksaan apakah persamaan benar-benar persamaan kuadrat. Program akan membaca syarat pada kondisi True dimana ketika nilai

		a == 0 program akan menampilkan keterangan bahwa persamaan tersebut bukan persamaan kuadrat.
6	Print(“bukan persamaan kuadrat”)	Program akan menampilkan teks setelah mengonfirmasi jika kondisi terpenuhi.
7	Else:	Menjadi penghubung ketika kondisi sebelumnya tidak terpenuhi. Ketika program melakukan proses pemeriksaan pada nilai a dan ternyata tidak memenuhi kondisi program akan berpindah untuk melakukan proses perhitungan diskriminan ketika kondisi tidak terpenuhi.
8	Diskriminan = $b^2 - 4ac$	Program melakukan perhitungan untuk memperoleh nilai diskriminan. Setelah terbukti bahwa kondisi asumsi tidak terpenuhi.
9	#pengecekan kondisi akar	Sebagai penanda bahwa proses yang sedang berjalan adalah memeriksa apakah persamaan kuadrat memiliki nilai akar-akar real.
10	If Diskriminan ≥ 0	Kode if digunakan untuk melakukan pemeriksaan apakah persamaan tersebut memiliki akar-akar real. Program akan membaca syarat pada kondisi True dimana ketika nilai Diskriminan ≥ 0 program melakukan proses perhitungan untuk mencaari nilai x1 dan x2 yang akan menjadi tempat menyimpan nilai akar-akar real.
11	$x1 = \frac{-b + \text{math.sqrt}(\text{Diskriminan})}{(2*a)}$	Setelah terbukti bahwa kondisi terpenuhi maka selanjutnya program akan melakukan proses perhitungan untuk mencari nilai akar

		real dari persamaan. Kode <code>math.sqrt</code> digunakan untuk memunculkan simbol ($\sqrt{}$) akaar karena untuk perhitungan sederhana pada python tidak memiliki simbol akar, oleh karena itu dibutuhkan modul <code>math</code> untuk memunculkan simbol tersebut. Setelah itu dilakukan proses perhitungan dengan program membaca setiap nilai yang sudah didefinisikan sebelumnya untuk memperoleh nilai <code>x1</code>. Konsep kerja kode ini sama dengan ketika akan mencari <code>x2</code> hanya saja simbol <code>+</code> berganti menjadi <code>-</code>.
12	#menampilkan hasil dengan format 3 desimal	Kode memberikan tanda bahwa proses ini akan menampilkan hasil dari perhitungan sebelumnya
13	<code>Print(f'\nPersamaan memiliki akar-akar real:')</code>	Sebelum menampilkan teks, program akan membaca penggunaan fitur f-string untuk melakukan format teks. Dan <code>\n</code> akan membuat kursor turun satu baris dan akhirnya program membaca perintah <code>print</code> untuk menampilkan teks.
14	<code>Print(f'x1 = {x1:.3f}')</code>	Sebelum menampilkan teks, program akan terlebih dahulu mengatur nilai <code>x1</code> dengan format <code>.3f</code> yaitu tampilan angka menjadi 3 angka dibelakang koma. Setelah itu fitur f string membentuk format teks dan program menampilkan teks beserta nilai <code>x1</code> . Alur program ini juga berjalan sama untuk menampilkan nilai <code>x2</code> .
15	<code>elif Diskriminan == 0:</code>	Ketika program sudah membaca bahwa kondisi pertama tidak terpenuhi, program akan melakukan pemeriksaan kondisi kedua

		dimana jika nilai diskriminan = 0 maka persamaan memiliki akar kembar.
16	$x1 = -b / (2*a)$	Setelah terbukti bahwa kondisi terpenuhi, program akan melakukan perhitungan untuk mencari nilai akar kembar, alur program ini juga sama ketika program akan mencari x2 dari persamaan.
17	Print(f"\nPersamaan memiliki akar kembar:")	Sebelum menampilkan teks, program akan membaca penggunaan fitur f-string untuk melakukan format teks. Dan \n akan membuat kursor turun satu baris dan akhirnya program membaca perintah print untuk menampilkan teks.
18	Print(f'x1 = {x1 :.3f}')	Sebelum menampilkan teks, program akan terlebih dahulu mengatur nilai x1 dengan format .3f yaitu tampilan angka menjadi 3 angka dibelakang koma. Setelah itu fitur f string membentuk format teks dan program menampilkan teks beserta nilai x1. Alur program ini juga berjalan sama untuk menampilkan nilai x2.
19	Else:	Menjadi penghubung antara kondisi ketika keadaan terpenuhi dan tidak. Ketika program sudah mendeteksi bahwa kondisi tidak terpenuhi, proses akan berpindah pada kondisi tidak terpenuhi dimana akan menampilkan bahwa persamaan hanya memiliki akar imajiner
20	Print("\nPersamaan hanya memiliki akar-akar imajiner")	\n akan membuat kursor turun satu baris lalu program akan membaca perintah untuk menampilkan teks

NO	BARIS KODE R	PENJELASAN
1	#mencari akar-akar persamaan kuadrat	Digunakan sebagai dokumentasi judul program yang tidak berpengaruh pada proses perhitungan.
2	#mendefinisikan koefisien	Digunakan sebagai tanda program masuk pada tahap mendefinisikan nilai koefisien.
3	a <- as.numeric(readline(prompt="masukkan nilai a: "))	Sebelum menyimpan nilai variable program akan memunculkan teks di konsol, lalu pada kode readline program menunggu pengguna memasukkan ketentuan berupa nilai a yaitu koefisien dari x^2 cara menginput nilai tersebut yaitu klik enter setelah tutup kurung terakhir pada kode tersebut lalu ketik nilai yang ingin dimasukkan berupa bilangan bulat. Pada kode as.numeric itu akan mengubah tipe data menjadi angka karena pada saat kode dibaca oleh readline angka akan dibaca sebagai string/karakter. Setelah itu baru nilai akan disimpan ke variable. Kode ini memiliki alur yang sama dengan mendefinisikan variable b dan c. data ini dibutuhkan untuk mencari nilai diskriminan yang akan menjadi penentu apakah persamaan memiliki akar-akar real atau tidak, lalu data berupa nilai a, b, dan c akan digunakan dalam proses perhitungan untuk mencari nilai akar-akar dari persamaan.

4	#mengidentifikasi persamaan kuadrat	Menandai judul proses validasi apakah persamaan tersebut benar-benar persamaan kuadrat.
5	If (a == 0) {	Program melakukan pemeriksaan dengan sebuah kondisi bersyarat a == 0 (ketika nilai a sama dengan 0). Syarat ini digunakan untuk memastikan bahwa persamaan tersebut benar-benar persamaan kuadrat karena a merupakan koefisien milik x^2 dimana x^2 melambangkan bahwa persamaan tersebut merupakan persamaan kuadrat.
6	Cat("persamaan bukan persamaan kuadrat\n")	Menampilkan keterangan bahwa persamaan tersebut bukan persamaan kuadrat dan berpindah ke baris baru
7	} else {	Medefinisikan ketika kondisi sebelumnya tidak terpenuhi.
8	Diskriminan <- $b^2 - 4*a*c$	Melakukan proses perhitungan untuk memperoleh nilai diskriminan yang menjadi penentu apakah persamaan memiliki akar-akar real, akar kembar, atau hanya memiliki akar-akar imajiner.
9	#mencari akar-akar persamaan	Menandai judul proses mencari nilai akar-akar real dari nilai
10	If (Diskriminan > 0) {	Program melakukan pemeriksaan bersyarat Diskriminan > 0 (ketika nilai diskriminan lebih besar dari 0). Ini digunakan untuk mengetahui apakah persamaan tersebut memiliki akar-akar real.

11	<code>x1 <- (-b + sqrt(Diskriminan) / (2*a)</code>	Melakukan proses perhitungan untuk memperoleh nilai akar-akar real. Begitu juga dengan x2.
12	<code>Cat(sprintf("akar-akar real adalah: x1 = %.3f dan x2 = %.3f\n", x1, x2))</code>	Menampilkan nilai dari x1 dan x2. Sprintf() digunakan agar program menampilkan akar-akar bilangan real dalam 3 desimal, dengan simbol %.3f yang artinya dalam menampilkan nilai x1 dan x2 hanya akan dibuat menjadi 3 angka dibelakang koma.
13	<code>} else if (Diskriminan = 0) {</code>	Ketika program sudah membaca bahwa kondisi pertama tidak terpenuhi, proses dari kondisi satu akan dilewati, kemudian masuk ke kondisi kedua dimana program akan memeriksa apakah nilai diskriminan pada persamaan itu = 0. Ketika kondisi terpenuhi, program akan menampilkan bahwa persamaan memiliki akar kembar.
14	<code>x1 <- -b / (2*a)</code>	Setelah program selesai melakukan pemeriksaan dan kondisi 2 terpenuhi program akan melakukan proses perhitungan untuk memperoleh nilai akar dari persamaan. Begitu juga dengan x2.
15	<code>cat(sprintf("persamaan memiliki akar kembar : x1 = %.3f dan x2 = %.3f\n", x1, x2))</code>	Menampilkan nilai dari x1 dan x2. Sprintf() digunakan agar program menyusun format teks nilai akar dalam 3 desimal, dengan simbol %.3f yang artinya dalam menampilkan nilai x1 dan x2 hanya akan dibuat menjadi 3 angka dibelakang koma.
16	<code>} else {</code>	Medefinisikan ketika kondisi sebelumnya tidak terpenuhi.

17	Cat(“persamaan hanya mempunyai akar-akar imajiner\n”)	Menampilkan teks bahwa persamaan hanya mempunyai akar-akar imajiner karena ketika Diskriminan bernilai negatif nilai didalam akar akan bernilai negated. Oleh karena itu, nilai akar yang dihasilkan akan berupa imajiner.
----	---	--

5.4 Program 4

Berikut adalah penjelasan line-by-line code program 4 :

NO	BARIS KODE PYTHON	PENJELASAN
1	nip = input("Masukkan NIP ASN: ")	Membaca NIP yang diketik pengguna dan menyimpannya sebagai string ke variabel nip
2	if len(nip) != 18 or not nip.isdigit():	Memvalidasi bahwa NIP memiliki tepat 18 karakter dan seluruhnya berupa angka; salah satu syarat gagal maka program menampilkan error
3	tahun = nip[0:4]	Mengambil 4 karakter pertama NIP (indeks 0–3) sebagai representasi tahun lahir
4	bulan = int(nip[4:6])	Mengambil karakter ke-5 dan ke-6 NIP (indeks 4–5), lalu dikonversi ke integer sebagai nomor bulan lahir
5	tanggal = int(nip[6:8])	Mengambil karakter ke-7 dan ke-8 NIP (indeks 6–7), lalu dikonversi ke integer sebagai tanggal lahir
6	if bulan < 1 or bulan > 12:	Memvalidasi bahwa nilai bulan berada pada rentang yang sah (1–12)
7	if bulan in [1, 3, 5, 7, 8, 10, 12]:	Mengecek apakah bulan termasuk kelompok bulan yang memiliki 31 hari
8	elif bulan in [4, 6, 9, 11]:	Mengecek apakah bulan termasuk kelompok bulan yang memiliki 30 hari

9	<code>elif bulan == 2: maks_hari = 29</code>	Menetapkan batas hari Februari sebagai 29 (mengakomodasi kemungkinan tahun kabisat tanpa perlu memeriksa tahun)
10	<code>if tanggal < 1 or tanggal > maks_hari:</code>	Memvalidasi bahwa tanggal tidak melebihi jumlah hari maksimum bulan yang bersangkutan
11	<code>if bulan == 1: nama_bulan = "Januari"</code> <code>s/d elif bulan == 12</code>	Serangkaian percabangan untuk mengonversi nomor bulan menjadi nama bulan dalam bahasa Indonesia, sesuai ketentuan soal
12	<code>print(f"Tanggal lahir: {tanggal}</code> <code>{nama_bulan} {tahun}")</code>	Menampilkan hasil akhir berupa tanggal lahir ASN dalam format hari, nama bulan, dan tahun

NO	BARIS KODE R	PENJELASAN
1	<code>nip <- readline("Masukkan NIP ASN: ")</code>	Membaca input NIP dari pengguna secara interaktif dan menyimpannya sebagai string
2	<code>if (nchar(nip) != 18 grepl("[^0-9]", nip))</code>	Memvalidasi panjang NIP (18 karakter) dan memastikan tidak ada karakter non-digit menggunakan ekspresi reguler
3	<code>tahun <- substr(nip, 1, 4)</code>	Mengambil substring karakter posisi 1–4 sebagai tahun lahir (R menggunakan indeks berbasis 1)
4	<code>bulan <- as.integer(substr(nip, 5, 6))</code>	Mengambil substring posisi 5–6 dan mengonversinya ke integer sebagai nomor bulan lahir
5	<code>tanggal <- as.integer(substr(nip, 7, 8))</code>	Mengambil substring posisi 7–8 dan mengonversinya ke integer sebagai tanggal lahir
6	<code>if (bulan < 1 bulan > 12)</code>	Memvalidasi bahwa bulan berada dalam rentang yang valid

7	<code>if (bulan %in% c(1,3,5,7,8,10,12))</code>	Mengecek keanggotaan bulan dalam vektor bulan ber-31-hari menggunakan operator <code>%in%</code> khas R
8	<code>maks_hari <- 29</code>	Menetapkan batas hari untuk Februari
9	<code>if (tanggal < 1 tanggal > maks_hari)</code>	Memvalidasi tanggal terhadap batas hari bulan yang bersangkutan
10	<code>if (bulan == 1) nama_bulan <- "Januari"</code> <code>s/d else if (bulan == 12)</code>	Percabangan berantai untuk mengonversi nomor bulan ke nama bulan bahasa Indonesia
11	<code>cat(sprintf("Tanggal lahir: %d %s %s\n", tanggal, nama_bulan, tahun))</code>	Mencetak output terformat ke konsol; <code>%d</code> untuk integer tanggal, <code>%s</code> untuk string nama bulan dan tahun

5.5 Program 5

Berikut adalah penjelasan line-by-line code program 5 :

NO	BARIS KODE PYTHON	PENJELASAN
1.	<code>Import math</code>	Mengimpor modul math untuk mengakses fungsi <code>sqrt()</code>
2.	<code>def hitung_jarak(titik_U,pusat_cluster)</code>	Mendefinisikan fungsi yang menerima dua titik 3 dimensi sebagai parameter
3.	<code>x1, x2, x3=titik_U</code>	Membagi tuple titik U menjadi tiga variabel koordinat terpisah
4.	<code>y1, y2, y3=pusat_cluster</code>	Membagi tuple pusat_cluster menjadi tiga variabel koordinat terpisah, yang mewakili koordinat pusat cluster
5.	<code>jarak = math.sqrt(</code> <code>(x1-y1)**2 +</code> <code>(x2-y2)**2 +</code> <code>(x3-y3)**2</code>	Menerapkan rumus jarak Euclidean 3 dimensi sesuai pada soal

	)	
6.	return jarak	Mengembalikan nilai jarak yang sudah dihitung agar bisa digunakan di luar fungsi ini
7.	def klasifikasi_cluster(titik_U):	Membuat fungsi bernama klasifikasi_cluster yang menerima satu data masukan yaitu titik_U, berisi koordinat titik yang ingin diklasifikasikan
8.	pusat_A = (2, 1, 3) pusat_B=(1,-4,6) pusat_C=(-2,3,-2)	Mendefinisikan koordinat pusat Cluster A, B, C sebagai tuple dengan konstanta tuple
9.	jarak_A=hitung_jarak(titik_U,pusat_A) jarak_B=hitung_jarak(titik_U,pusat_B) jarak_C=hitung_jarak(titik_U,pusat_C)	Memanggil fungsi untuk menghitung jarak dari U ke pusat A, B, dan C
10.	if jarak_A<jarak_B and jarak_A<jarak_C: cluster="A"	Percabangan: Jika A memiliki jarak paling dekat dari ketiganya, maka merupakan cluster "A"
11.	elif jarak_B<jarak_A and jarak_B<jarak_C: cluster="B"	Percabangan: Jika kondisi pertama tidak terpenuhi, cek apakah B memiliki jarak paling dekat dari ketiganya, Jika benar maka merupakan cluster "B"
12.	elif jarak_C<jarak_A and jarak_C<jarak_B: cluster="C"	Percabangan: Jika kondisi kedua tidak terpenuhi, cek apakah C memiliki jarak paling dekat dari ketiganya, Jika benar maka merupakan cluster "C"
	else: cluster="Tidak dapat ditentukan"	Percabangan:Jika ketiganya tidak terpenuhi, maka cluster tidak dapat ditentukan

13.	<code>return cluster,jarak_A,jarak_B,jarak_C</code>	Mengembalikan empat nilai sekaligus: nama cluster hasil klasifikasi dan ketiga nilai jarak
14.	<code>x1=float(input("Masukkan x1:")) x2=float(input("Masukkan x2:")) x3=float(input("Masukkan x3:"))</code>	Memasukkan nilai x1, x2, x3 sebagai scenario uji
15.	<code>titik_U=(x1,x2,x3)</code>	Menyimpan koordinat input sebagai tuple sesuai ketentuan pada soal
16.	<code>cluster, jarak_A, jarak_B, jarak_C = klasifikasi_cluster(titik_u)</code>	Memecah empat nilai kembaliannya ke empat variabel terpisah sekaligus
17.	<code>print(f"Titik U({x1}, {x2}, {x3})")</code>	Menampilkan ketiga koordinat pada titik U
18.	<code>print(f"Jarak ke Cluster A:{jarak_A:.4f}") print(f"Jarak ke Cluster B:{jarak_B:.4f}") print(f"Jarak ke Cluster C:{jarak_C:.4f}")</code>	Menampilkan jarak ke tiap cluster (A, B, C) dengan 4 desimal
	<code>print(f"Titik U tergolong Cluster {cluster}")</code>	Mencetak ulang koordinat titik U yang diinput sebagai konfirmasi

NO	BARIS KODE R	PENJELASAN
1.	<code>hitung_jarak<- function(titik_U,pusat_cluster)</code>	Menyimpan nilai ke variabel. Fungsi ini menerima dua data masukan: koordinat titik U dan koordinat pusat cluster
2.	<code>x1<-titik_u[1];y1<-pusat_cluster[1] x2<-titik_u[2];y2<-pusat_cluster[2] x3<-titik_u[3];y3<-pusat_cluster[3]</code>	X1, x2, x3: Mengambil elemen pertama. Kedua, dan ketiga dari titik_u dan menyimpannya ke x1, x2, dan x3 Y1, y2, y3: Mengambil elemen pertama, kedua, dan ketiga dari pusat_cluster dsn

		menyimpannya ke y1 sebagai koordinat pusat pertama pusat cluster, y2, dan y3
3.	<pre>jarak<-sqrt((x1-y1)^2+ (x2-y2)^2+ (x3-y3)^2)</pre>	Menghitung rumus jarak yaitu mengakar kuadrat sesuai dengan rumus pada soal
4.	<pre>return(jarak)</pre>	Mengembalikan nilai jarak yang sudah dihitung agar bisa dipakai di luar fungsi
5.	<pre>klasifikasi_cluster<-function(titik_U){</pre>	Menerima satu data masukan yaitu titik_u yang berisi koordinat titik yang akan diklasifikasikan
6.	<pre>pusat_A<-c(2,1,3) pusat_B<-c(1,-4,6) pusat_C<-c(-2,3,-2)</pre>	Membuat vector berisi tiga angka koordinat pusat Cluster A, B, dan C dengan nilai konstanta sesuai pada soal
7.	<pre>jarak_A<-hitung_jarak(titik_U,pusat_A) jarak_B<-hitung_jarak(titik_U,pusat_B) jarak_C<-hitung_jarak(titik_U,pusat_C)</pre>	Menghitung jarak dari titik U ke pusat Cluster A, B, dan C
8.	<pre>if(jarak_A<jarak_B&&jarak_A<jarak_C){ cluster<-"A"</pre>	Mengecek apakah jarak ke A lebih kecil dari jarak ke B sekaligus lebih kecil dari jarak ke C. Jika kondisi terpenuhi, termasuk ke dalam cluster "A"
9.	<pre>}else if(jarak_B<jarak_A&&jarak_B<jarak_C){ cluster<-"B"</pre>	Jika kondisi pertama tidak terpenuhi, cek apakah jarak ke B lebih kecil dari jarak ke A dan lebih kecil dari jarak ke C. Jika kondisi terpenuhi, termasuk ke dalam cluster "B"
10.	<pre>}else if(jarak_C<jarak_A&&jarak_C<jarak_B){ cluster<-"B"</pre>	Jika kondisi kedua tidak terpenuhi, cek apakah jarak ke C lebih kecil dari jarak ke A dan lebih kecil dari jarak ke B. Jika

		kondisi terpenuhi, termasuk ke dalam cluster “C”
11.	<pre> }else{ cluster<-"tidak dapat ditentukan" } </pre>	Jika ketiga kondisi di atas tidak terpenuhi, maka cluster tidak dapat ditentukan
12.	<pre> return(list(cluster=cluster, jarak_A=jarak_A, jarak_B=jarak_B, jarak_C=jarak_C)) } </pre>	mengembalikan hasil fungsi dalam bentuk list Memasukkan variabel cluster ke dalam list dengan nama cluster. Memasukkan variable cluster jarak_A, B, C ke dalam list dengan nama jarak_A, B, C
13.	<pre> x1<-as.numeric(readline("Masukkan x1: ")) x2<-as.numeric(readline("Masukkan x2: ")) x3<-as.numeric(readline("Masukkan x3: ")) </pre>	Menghitung jarak dari titik U ke pusat Cluster A, B, dan C
14.	<pre> titik_U<-c(x1,x2,x3) </pre>	Menggabungkan ketiga koordinat menjadi satu vector.
15.	<pre> hasil<-klasifikasi_cluster(titik_U) </pre>	menyimpan seluruh hasil kembaliannya (berupa list) ke variable hasil
16.	<pre> cat(sprintf("\nTitik U(%.4g,%.4g,%.4g)\n",x1,x2,x3)) </pre>	Mencetak ulang koordinat titik U
17.	<pre> cat(sprintf("Jarak ke Cluster A:%.4f\n",hasil\$jarak_A)) cat(sprintf("Jarak ke Cluster B:%.4f\n",hasil\$jarak_B)) cat(sprintf("Jarak ke Cluster C:%.4f\n",hasil\$jarak_C)) </pre>	Mencetak jarak ke Cluster A, B, dan C dengan 4 angka di belakang koma.

18.	<code>cat(sprintf("\nTitik U tergolong Cluster %s\n",hasil\$cluster))</code>	Mencetak kesimpulan akhir klasifikasi apakah termasuk cluster A, B, atau C
-----	--	--

5.6 Program 6

Berikut adalah penjelasan line-by-line code program 6 :

NO.	BARIS KODE PYTHON	PENJELASAN
1	<code>import math</code>	Mengimpor modul math agar program dapat menggunakan fungsi matematika, seperti akar kuadrat (sqrt).
2	<code>def hitung_interval(p_hat, n, alpha):</code>	Mendefinisikan fungsi bernama <code>hitung_interval</code> yang menerima proporsi sampel (<code>p_hat</code>), ukuran sampel (<code>n</code>), dan tingkat signifikansi (<code>alpha</code>) sebagai parameter.
3	<code>if alpha == 0.05:</code>	Memeriksa apakah nilai <code>alpha</code> sebesar 0,05.
4	<code>z = 1.96</code>	Menetapkan nilai <code>z</code> sebesar 1,96 yang digunakan untuk <code>alpha</code> 0,05.
5	<code>elif alpha == 0.10:</code>	Memeriksa apakah nilai <code>alpha</code> sebesar 0,10.
6	<code>z = 1.645</code>	Menetapkan nilai <code>z</code> sebesar 1,645 yang digunakan untuk <code>alpha</code> 0,10.
7	<code>margin_error = z * math.sqrt((p_hat * (1 - p_hat)) / n)</code>	Menghitung margin of error berdasarkan proporsi sampel, ukuran sampel, dan nilai <code>z</code> .
8	<code>batas_bawah = p_hat - margin_error</code>	Menghitung batas bawah interval konfidensi.
9	<code>batas_atas = p_hat + margin_error</code>	Menghitung batas atas interval konfidensi.
10	<code>return {"BB": batas_bawah, "BA": batas_atas}</code>	Mengembalikan hasil perhitungan dalam bentuk dictionary yang berisi batas bawah dan batas atas.

11	<code>p_hat = ("Input nilai p_hat")</code>	Menginput nilai proporsi sampel yang digunakan dalam perhitungan.
12	<code>n = ("Input nilai n")</code>	Menginput ukuran sampel yang digunakan dalam perhitungan.
13	<code>alpha = ("Input nilai alpha")</code>	Menginput tingkat signifikansi yang digunakan untuk menentukan nilai z.
14	<code>if p_hat < 0 or p_hat > 1:</code>	Memvalidasi bahwa proporsi sampel berada pada rentang 0 sampai 1.
15	<code>print("Error: Proporsi sampel harus berada pada rentang 0 sampai 1")</code>	Menampilkan pesan kesalahan jika nilai proporsi sampel tidak valid.
16	<code>elif n <= 0:</code>	Memvalidasi bahwa ukuran sampel lebih besar dari nol.
17	<code>print("Error: Ukuran sampel harus lebih dari 0")</code>	Menampilkan pesan kesalahan jika ukuran sampel tidak valid.
18	<code>elif alpha != 0.05 and alpha != 0.10:</code>	Memvalidasi bahwa alpha harus bernilai 0,05 atau 0,10.
19	<code>print("Error: Alpha harus bernilai 0.05 atau 0.10")</code>	Menampilkan pesan kesalahan jika nilai alpha tidak sesuai ketentuan.
20	<code>else: hasil = hitung_interval(p_hat, n, alpha)</code>	Memanggil fungsi <code>hitung_interval</code> untuk menghitung interval konfidensi dan menyimpan hasilnya ke variabel "hasil".
21	<code>print("Batas Bawah =", hasil["BB"])</code>	Menampilkan nilai batas bawah interval konfidensi.
22	<code>print("Batas Atas =", hasil["BA"])</code>	Menampilkan nilai batas atas interval konfidensi.
23	<code>print("Interval konfidensi = (", round(hasil["BB"], 4), ",", round(hasil["BA"], 4), ")")</code>	Menampilkan interval konfidensi dalam format pasangan batas bawah dan batas atas dengan pembulatan 4 angka desimal.

NO.	BARIS KODE R	PENJELASAN
1	<code>hitung_interval <- function(p_hat, n, alpha) {</code>	Mendefinisikan fungsi <code>hitung_interval</code> yang menerima proporsi sampel (<code>p_hat</code>),

		ukuran sampel (n), dan tingkat signifikansi (alpha) sebagai parameter.
2	if (alpha == 0.05) {	Memeriksa apakah nilai alpha sebesar 0,05.
3	z <- 1.96	Menetapkan nilai z sebesar 1,96 untuk alpha 0,05.
4	} else if (alpha == 0.10) {	Memeriksa apakah nilai alpha sebesar 0,10.
5	z <- 1.645	Menetapkan nilai z sebesar 1,645 untuk alpha 0,10.
6	margin_error <- z * sqrt((p_hat * (1 - p_hat)) / n)	Menghitung margin of error berdasarkan proporsi sampel, ukuran sampel, dan nilai z.
7	batas_bawah <- p_hat - margin_error	Menghitung batas bawah interval konfidensi.
8	batas_atas <- p_hat + margin_error	Menghitung batas atas interval konfidensi.
9	return(list(BB = batas_bawah, BA = batas_atas))	Mengembalikan hasil perhitungan dalam bentuk list yang berisi batas bawah dan batas atas.
10	p_hat <- ("Input nilai p_hat")	Menginput nilai proporsi sampel yang digunakan dalam perhitungan.
11	n <- ("Input nilai n")	Menginput ukuran sampel yang digunakan dalam perhitungan.
12	alpha <- ("Input nilai alpha")	Menginput tingkat signifikansi yang digunakan untuk menentukan nilai z.
13	if (p_hat < 0 p_hat > 1) {	Memvalidasi bahwa proporsi sampel berada pada rentang 0 sampai 1.
14	cat("Error: p_hat harus berada antara 0 dan 1\n")	Menampilkan pesan kesalahan apabila proporsi sampel tidak valid.
15	} else if (n <= 0) {	Memvalidasi bahwa ukuran sampel lebih besar dari nol.

16	<code>cat("Error: ukuran sampel harus lebih dari 0\n")</code>	Menampilkan pesan kesalahan apabila ukuran sampel tidak valid.
17	<code>} else if (alpha != 0.05 && alpha != 0.10) {</code>	Memvalidasi bahwa nilai alpha bernilai 0,05 atau 0,10.
18	<code>cat("Error: alpha harus 0.05 atau 0.10\n")</code>	Menampilkan pesan kesalahan apabila nilai alpha tidak sesuai ketentuan.
19	<code>} else {hasil <- hitung_interval(p_hat, n, alpha)</code>	Memanggil fungsi <code>hitung_interval</code> dan menyimpan hasilnya ke variabel "hasil".
20	<code>cat("Batas bawah =", hasil\$BB, "\n")</code>	Menampilkan nilai batas bawah interval konfidensi.
21	<code>cat("Batas atas =", hasil\$BA, "\n")</code>	Menampilkan nilai batas atas interval konfidensi.
22	<code>cat("Interval konfidensi = (", round(hasil\$BB, 4), ", ", round(hasil\$BA, 4), ")\n") }</code>	Menampilkan interval konfidensi dalam format pasangan batas bawah dan batas atas dengan pembulatan 4 angka desimal.

BAB VI

PENGUJIAN PROGRAM

6.1 Program 1

1. Tabel pengujian

No	Input	Output yang diharapkan	Output pada Python	Output pada R	Status	Catatan
1	Kenaikan harga bensin yang terjadi belakangan ini membuat masyarakat mengeluhkan meningkatnya biaya transportasi dan kebutuhan sehari-hari. Banyak pengendara merasa pengeluaran mereka menjadi lebih besar dibandingkan sebelumnya. Kondisi ini juga berdampak pada naiknya harga beberapa kebutuhan pokok di pasaran.	Jumlah kalimat = 3 Jumlah kata = 38	Teks tersebut memuat 3 kalimat dan 38 kata.	Teks tersebut memuat 3 kalimat dan 38 kata.	Berhasil	Program berjalan sesuai harapan.
2	Harga BBM jenis Pertamax mengalami kenaikan dari Rp12.300 menjadi Rp16.250 per liter, sedangkan Paltalite tetap bertahan di harga sebelumnya. Kenaikan	Jumlah kalimat = 5 Jumlah kata = 47	Teks tersebut memuat 5 kalimat dan 47 kata.	Teks tersebut memuat 5 kalimat dan 47 kata.	Berhasil	Program berjalan sesuai harapan.

	ini terjadi akibat meningkatnya harga minyak dunia dan beban subsidi energi yang semakin besar. Kondisi tersebut membuat masyarakat khawatir karena biaya transportasi dan kebutuhan sehari-hari ikut meningkat.					
3	Perkembangan teknologi kecerdasan buatan atau Artificial Intelligence (AI) semakin pesat dan mulai digunakan dalam berbagai bidang, seperti pendidikan, kesehatan, dan komunikasi. Banyak perusahaan teknologi kini berlomba menciptakan inovasi baru agar pekerjaan manusia dapat dilakukan dengan lebih cepat dan efisien. Namun, sebagian masyarakat juga khawatir karena perkembangan	Jumlah kalimat = 3 Jumlah kata = 56	Teks tersebut memuat 3 kalimat dan 56 kata.	Teks tersebut memuat 3 kalimat dan 56 kata.	Berhasil	Program berjalan sesuai harapan.

	teknologi dapat mengurangi beberapa jenis lapangan pekerjaan di masa depan.					
--	---	--	--	--	--	--

2. Hasil running

The screenshot shows the RStudio interface. The console displays the following R code and its output:

```
#kondisi 1
> teks <- "Kenaikan harga bensin yang terjadi belakangan ini membuat masyarakat mengeluhkan meningkatnya biaya transportasi dan kebutuhan sehari-hari. Banyak pengendara merasa pengeluaran mereka menjadi lebih besar dibandingkan sebelumnya. Kondisi ini juga berdampak pada naiknya harga beberapa kebutuhan pokok di pasaran."
> jumlah_kata <- length(strsplit(teks, " ")[1])
> jumlah_kalimat <- length(regmatches(teks, gregexpr("\\.", teks)))
>
> cat("Teks tersebut memuat",
+     jumlah_kalimat,
+     "kalimat dan",
+     jumlah_kata,
+     "kata.")
Teks tersebut memuat 3 kalimat dan 38 kata.
>
```

Gambar 6.1 Input dan Output R Kondisi 2

The screenshot shows a Jupyter Notebook interface. The code cell contains the following Python code, and the output cell shows the result:

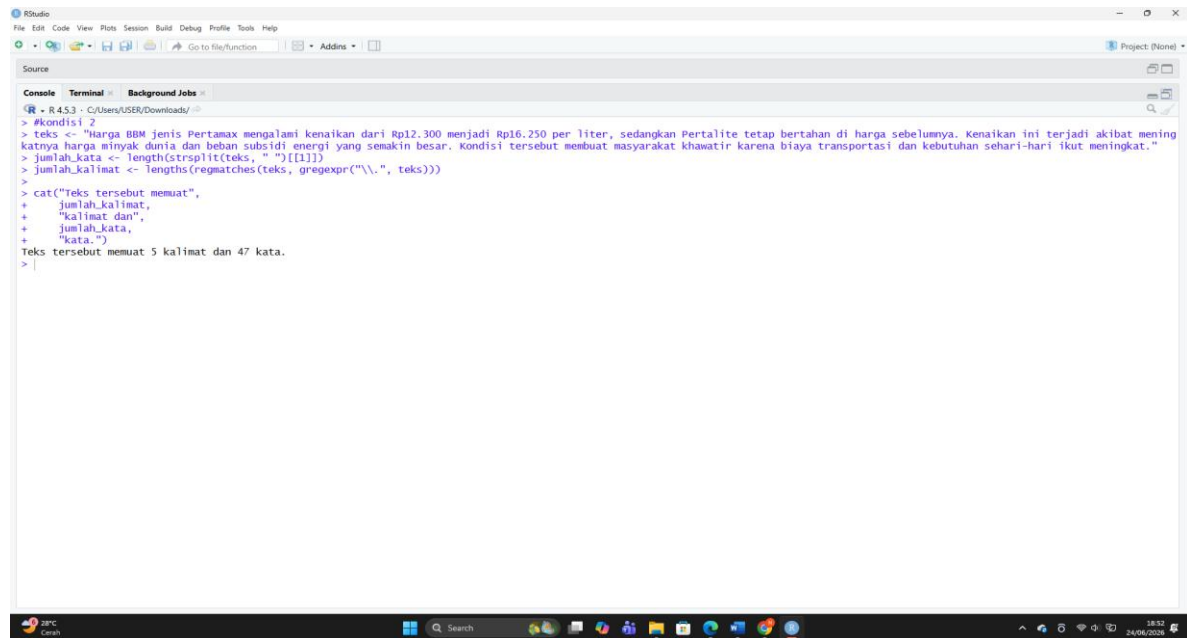
```
# kondisi 1
teks = "Kenaikan harga bensin yang terjadi belakangan ini membuat masyarakat mengeluhkan meningkatnya biaya transportasi dan kebutuhan sehari-hari. Banyak pengendara merasa pengeluaran mereka menjadi lebih besar dibandingkan sebelumnya. Kondisi ini juga berdampak pada naiknya harga beberapa kebutuhan pokok di pasaran."
jumlah_kata = len(teks.split())
jumlah_kalimat = teks.count(".")
print("Teks tersebut memuat", jumlah_kalimat, "kalimat dan", jumlah_kata, "kata.")
```

Output: Teks tersebut memuat 3 kalimat dan 38 kata.

Gambar 6.2 Input dan Output Python Kondisi 1

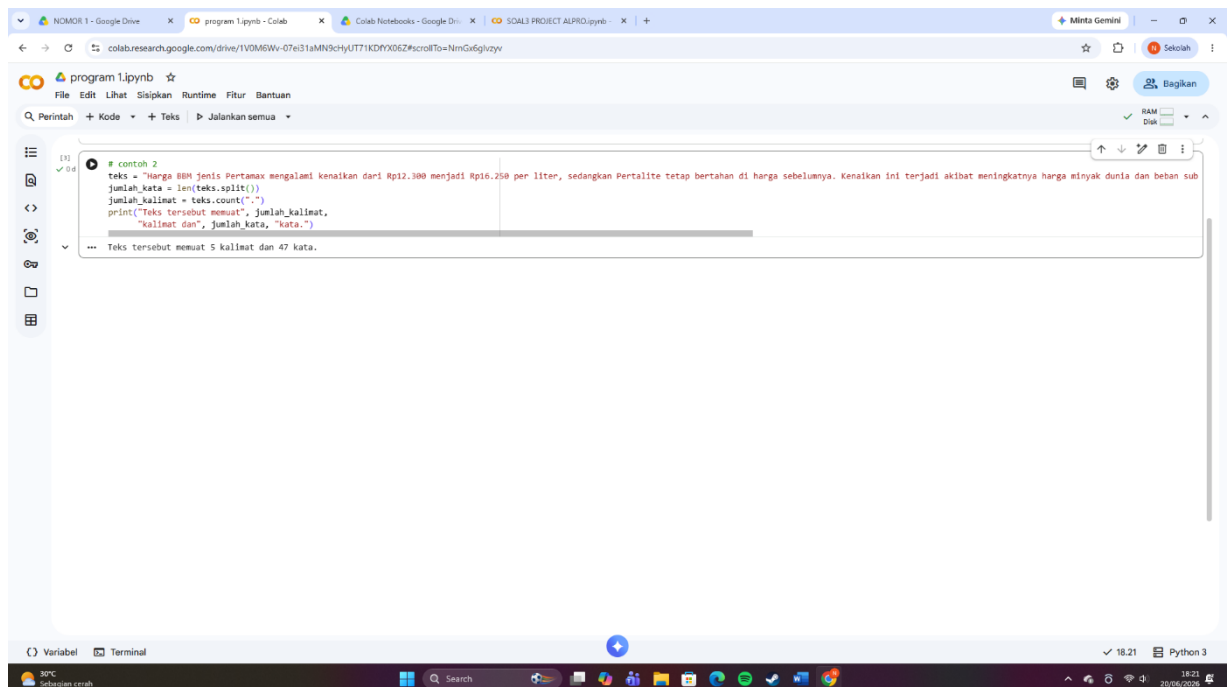
Interpretasi output :

Pada pengujian pertama, digunakan teks tentang kenaikan harga bensin. Program menghasilkan output berupa 3 kalimat dan 38 kata. Hasil yang diperoleh pada Python dan R sama dengan output yang diharapkan, sehingga program dinyatakan berhasil berjalan sesuai harapan.



```
> #kondisi 2
> teks <- "Harga BBM jenis Pertamina mengalami kenaikan dari Rp12.300 menjadi Rp16.250 per liter, sedangkan Pertamina tetap bertahan di harga sebelumnya. Kenaikan ini terjadi akibat meningkatnya harga minyak dunia dan beban subsidi energi yang semakin besar. Kondisi tersebut membuat masyarakat khawatir karena biaya transportasi dan kebutuhan sehari-hari ikut meningkat."
> jumlah_kata <- length(strsplit(teks, " ")[1])
> jumlah_kalimat <- length(regmatches(teks, gregexpr("\\.", teks)))
>
> cat("Teks tersebut memuat",
+     jumlah_kalimat,
+     "kalimat dan",
+     jumlah_kata,
+     "kata.")
Teks tersebut memuat 5 kalimat dan 47 kata.
>
```

Gambar 6.3 Input dan Output R Kondisi 2



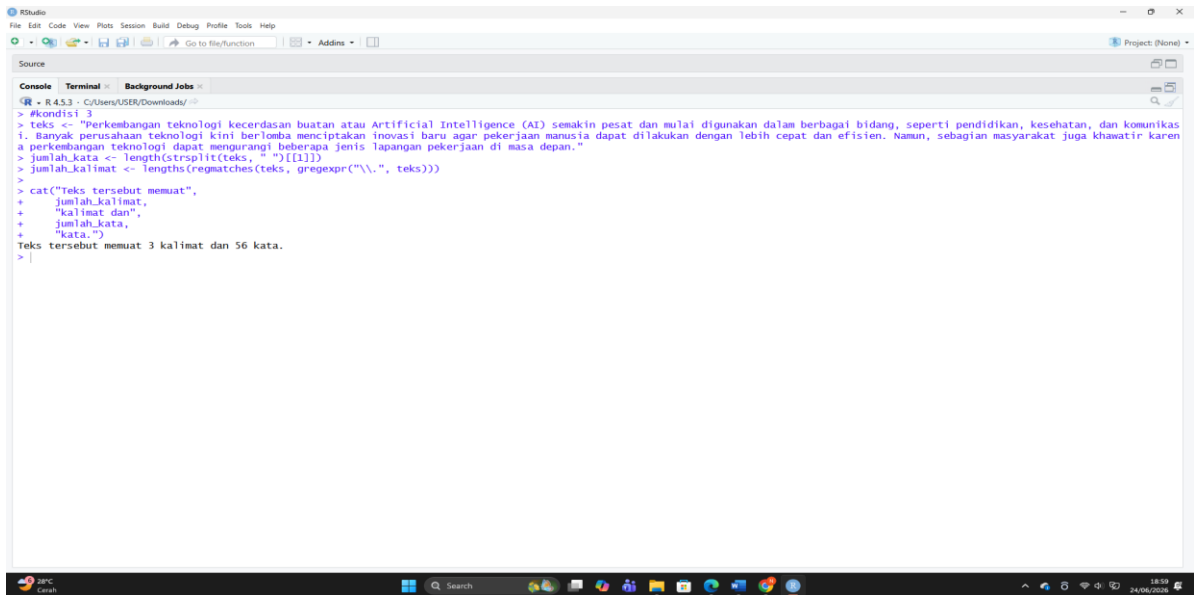
```
# contoh 2
teks = "Harga BBM jenis Pertamina mengalami kenaikan dari Rp12.300 menjadi Rp16.250 per liter, sedangkan Pertamina tetap bertahan di harga sebelumnya. Kenaikan ini terjadi akibat meningkatnya harga minyak dunia dan beban sub
jumlah_kalimat = len(teks.split())
jumlah_kata = teks.count(".")
print("Teks tersebut memuat", jumlah_kalimat,
      "kalimat dan", jumlah_kata, "kata.")

Teks tersebut memuat 5 kalimat dan 47 kata.
```

Gambar 6.4 Input dan Output Python Kondisi 3

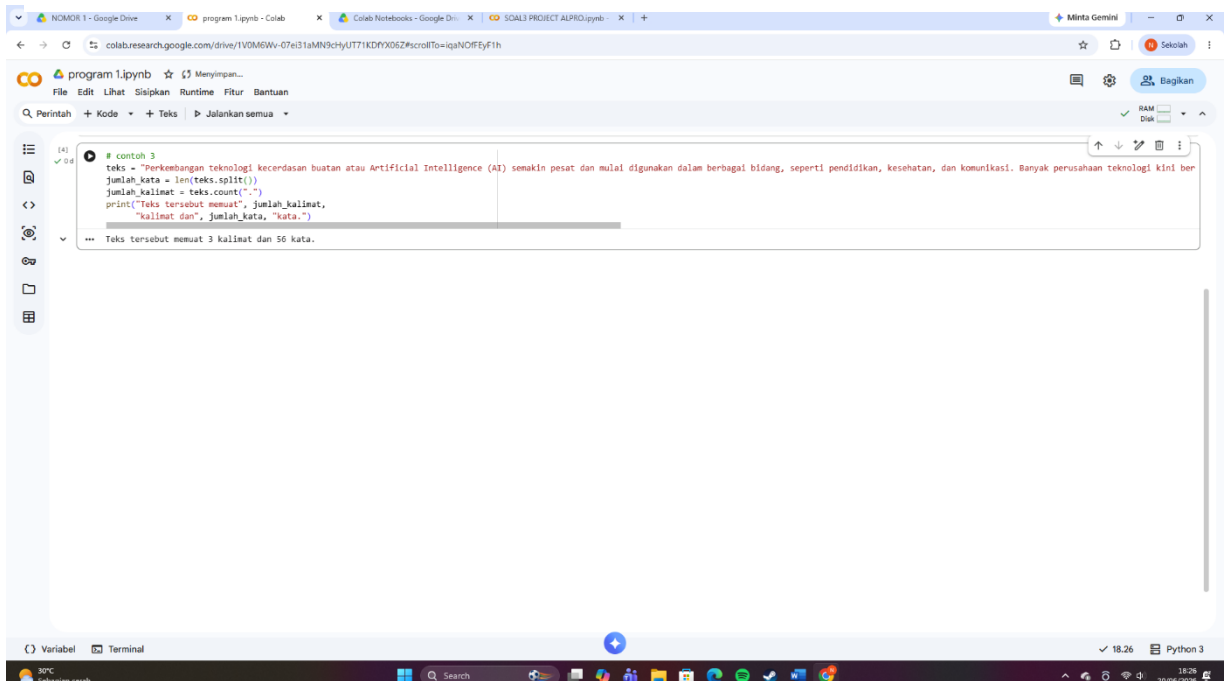
Interpretasi output :

Pada pengujian kedua, digunakan teks mengenai kenaikan harga BBM jenis Pertamax dan Pertalite. Program menghasilkan output berupa 5 kalimat dan 47 kata. Hasil yang ditampilkan pada Python dan R sesuai dengan output yang diharapkan sehingga program dapat menghitung jumlah kata dan kalimat dengan benar.



```
RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Source
Console Terminal Background Jobs
> #kondisi 3
> teks <- "Perkembangan teknologi kecerdasan buatan atau Artificial Intelligence (AI) semakin pesat dan mulai digunakan dalam berbagai bidang, seperti pendidikan, kesehatan, dan komunikasi. Banyak perusahaan teknologi kini berlomba menciptakan inovasi baru agar pekerjaan manusia dapat dilakukan dengan lebih cepat dan efisien. Namun, sebagian masyarakat juga khawatir karena perkembangan teknologi dapat mengurangi beberapa jenis lapangan pekerjaan di masa depan."
> jumlah_kata <- length(strsplit(teks, " ")[1])
> jumlah_kalimat <- length(regmatches(teks, greexpr("\\.", teks)))
>
> cat("Teks tersebut memuat",
+     jumlah_kalimat,
+     "kalimat dan",
+     jumlah_kata,
+     "kata.")
Teks tersebut memuat 3 kalimat dan 56 kata.
>
```

Gambar 6.5 Input dan Output R Kondisi 3



```
program 1.py
File Edit Lihat Sisipkan Runtime Fitur Bantuan
Perintah Kode Teks Jalankan semua
# contoh 3
teks = "Perkembangan teknologi kecerdasan buatan atau Artificial Intelligence (AI) semakin pesat dan mulai digunakan dalam berbagai bidang, seperti pendidikan, kesehatan, dan komunikasi. Banyak perusahaan teknologi kini berlomba menciptakan inovasi baru agar pekerjaan manusia dapat dilakukan dengan lebih cepat dan efisien. Namun, sebagian masyarakat juga khawatir karena perkembangan teknologi dapat mengurangi beberapa jenis lapangan pekerjaan di masa depan."
jumlah_kata = len(teks.split())
jumlah_kalimat = teks.count(".")
print("Teks tersebut memuat", jumlah_kalimat, "kalimat dan", jumlah_kata, "kata.")

Teks tersebut memuat 3 kalimat dan 56 kata.
```

Gambar 6.6 Output Python Kondisi 4

Interpretasi output :

Pada pengujian ketiga, digunakan teks tentang perkembangan teknologi kecerdasan buatan

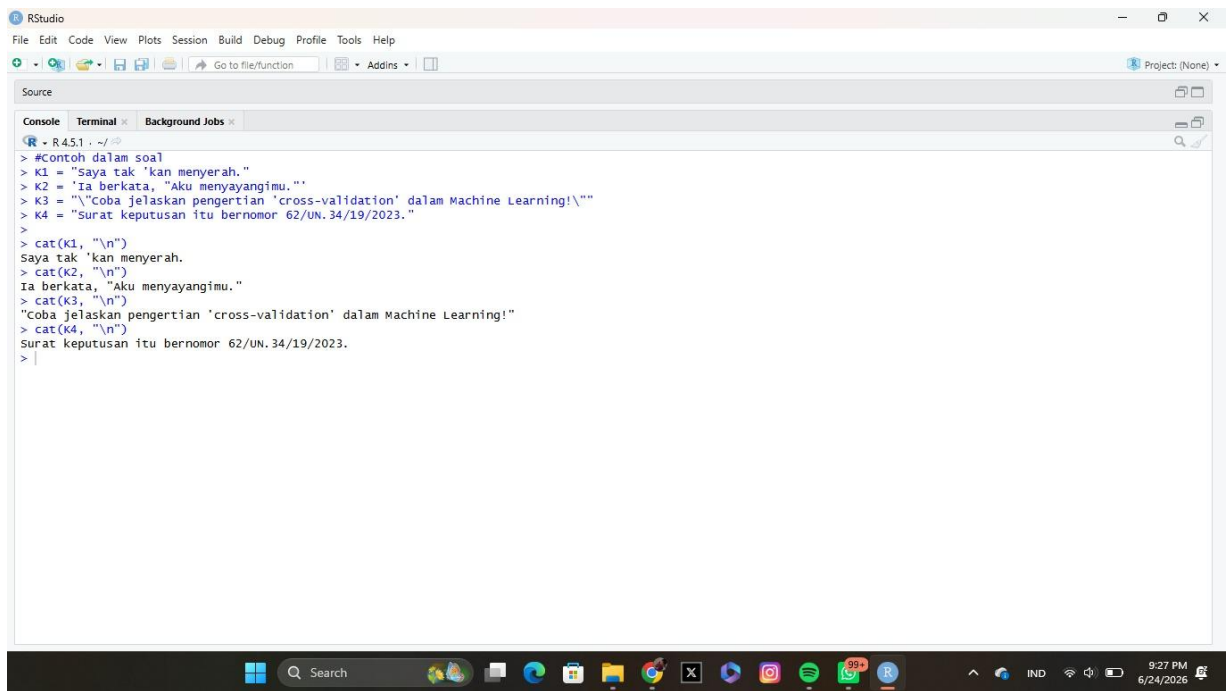
(AI). Program menghasilkan output berupa 3 kalimat dan 56 kata. Hasil yang diperoleh sesuai dengan output yang diharapkan sehingga program dinyatakan berhasil.

6.2 Program 2

1. Tabel pengujian

NO	Input	Output yang diharapkan	Output pada Python	Output pada R	Status	Catatan
1	K1 = "Saya tak 'kan menyerah."	Kalimat pertama tampil dengan benar	Saya tak 'kan menyerah.	Saya tak 'kan menyerah..	Berhasil	Program berjalan sesuai harapan.
2	K2 = 'Ia berkata, "Aku menyayangimu."'	Kalimat dengan tanda kutip tampil sesuai	Ia berkata, "Aku menyayangimu."'	Ia berkata, "Aku menyayangimu."'	Berhasil	Program berjalan sesuai harapan.
3	K3 = "\"Coba jelaskan pengertian 'cross-validation' ddalam Machine Learning! \\""	Kalimat dengan escape character tampil benar	"\"Coba jelaskan pengertian 'cross-validation' ddalam Machine Learning!'"	"\"Coba jelaskan pengertian 'cross-validation' ddalam Machine Learning!'"	Berhasil	Program berjalan sesuai harapan.
4	K4 = "Surat Keputusan itu bernomor 62/UN. 34/19/2023."	Kalimat keempat tampil sesuai	Surat Keputusan itu bernomor 62/UN. 34/19/2023.	Surat Keputusan itu bernomor 62/UN. 34/19/2023.	Berhasil	Program berjalan sesuai harapan.

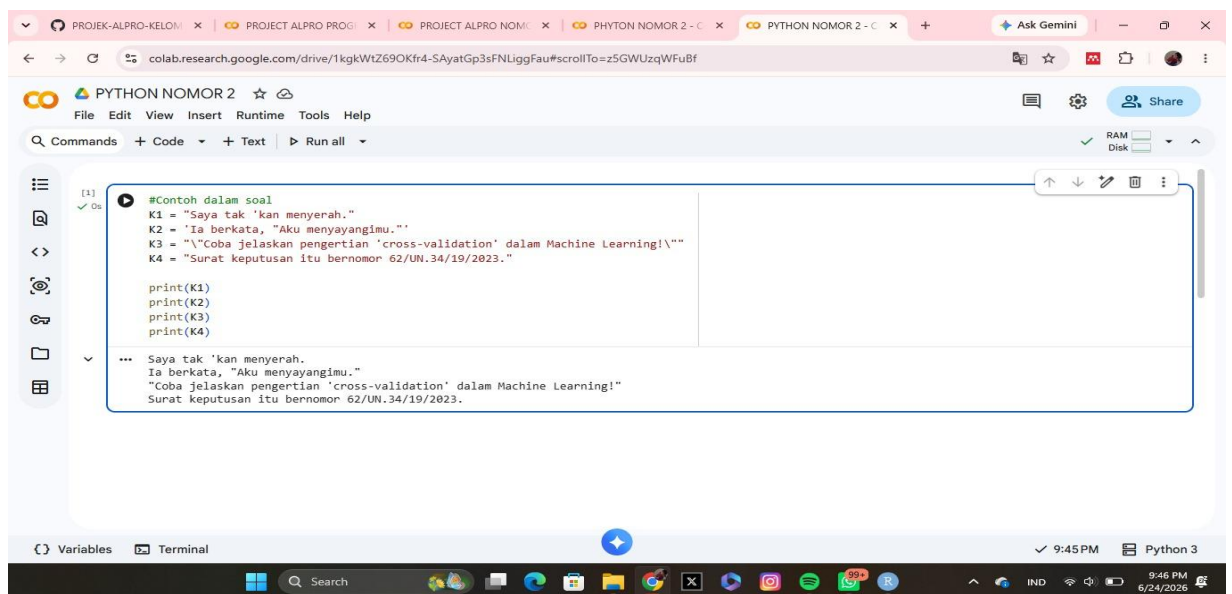
2. Hasil running



The screenshot shows the RStudio interface. The console window displays the following R code and its output:

```
> #Contoh dalam soal
> K1 = "Saya tak 'kan menyerah."
> K2 = "Ia berkata, "Aku menyayangimu.""
> K3 = "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
> K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."
>
> cat(K1, "\n")
Saya tak 'kan menyerah.
> cat(K2, "\n")
Ia berkata, "Aku menyayangimu."
> cat(K3, "\n")
"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!"
> cat(K4, "\n")
Surat keputusan itu bernomor 62/UN.34/19/2023.
> |
```

Gambar 6.7 Input dan Output R Kondisi 1



The screenshot shows the Google Colab interface. The code cell displays the following Python code and its output:

```
#Contoh dalam soal
K1 = "Saya tak 'kan menyerah."
K2 = "Ia berkata, "Aku menyayangimu.""
K3 = "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."

print(K1)
print(K2)
print(K3)
print(K4)

...
Saya tak 'kan menyerah.
Ia berkata, "Aku menyayangimu."
"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!"
Surat keputusan itu bernomor 62/UN.34/19/2023.
```

Gambar 6.8 Input dan Output Python Kondisi 1

Interpretasi output :

Pengujian dilakukan pada Python dan R dengan menggunakan data string yang sama. Pada Python, teks ditampilkan menggunakan fungsi `print()`, sedangkan pada R menggunakan `cat()`. Walaupun cara menampilkannya berbeda, kedua program menghasilkan output yang sama, yaitu seluruh kalimat dapat ditampilkan sesuai format yang diharapkan. Hasil ini menunjukkan bahwa baik Python maupun R dapat mengolah dan menampilkan string yang mengandung tanda petik serta karakter khusus dengan benar.

6.3 Program 3

1. Tabel pengujian

NO	Input	Output yang diharapkan	Output pada Python	Output pada R	Status	Catatan
1	a = 0, b = 4, c = 2	menampilkan teks : “persamaan bukan persamaan kuadrat”	menampilkan: “persamaan bukan persamaan kuadrat”	menampilkan : “bukan persamaan kuadrat”	berhasil	Program berjalan sesuai harapan
2	a = 1, b = -5, c = 6	menampilkan nilai akar-akar real : x1 = nilai dengan format 3 desimal dan x2 = nilai dengan format 3 desimal	menampilkan “akar-akar real adalah : x1 = 3,000 dan x2 = 2,000	menampilkan : “persamaan memiliki akar-akar real : x1 = 3,000 x2 = 2,000	berhasil	Program berjalan sesuai harapan
3	a = 1, b = 4, c = 4	menampilkan nilai akar kembar : x1 = nilai dengan format 3 desimal dan x2 = nilai dengan	menampilkan : “persamaan memiliki akar kembar : x1 = -2,000 dan x2 = -2,000	menampilkan : “persamaan memiliki akar kembar:” x1 = -2,000 x2 = -2,000	berhasil	Program berjalan sesuai harapan

		format 3 desimal				
4	a = 15, b = 4, c = 7	menampilkan teks “persamaan hanya mempunyai akar-akar imajiner”	menampilkan : “persamaan mempunyai akar-akar imajiner”	menampilkan : “Persamaan hanya memiliki akar-akar imajiner”	berhasil	Program berjalan sesuai harapan

2. Hasil running

```

RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Go to file/function Addins
Project: (None)

Source
Console Terminal Background Jobs
R 4.5.3 - C:/Users/USER/Downloads/
> #kondisi 1
> #mendefinisikan variabel
> a <- as.numeric(readline(prompt = "masukkan nilai a: "))
masukkan nilai a: 0
> b <- as.numeric(readline(prompt = "masukkan nilai b: "))
masukkan nilai b: 4
> c <- as.numeric(readline(prompt = "masukkan nilai c: "))
masukkan nilai c: 2
>
> #mengidentifikasi persamaan kuadrat
> if (a == 0) {
+   cat("persamaan bukan persamaan kuadrat\n")
+ } else {
+   Diskriminan <- b^2 - 4*a*c
+ }
persamaan bukan persamaan kuadrat
>
> #mencari akar-akar persamaan
> if (Diskriminan > 0) {
+   x1 <- (-b + sqrt(Diskriminan)) / (2*a)
+   x2 <- (-b - sqrt(Diskriminan)) / (2*a)
+   cat(sprintf("akar-akar real adalah: x1 = %.3f dan x2 = %.3f\n", x1, x2))
+ } else if (Diskriminan == 0) {
+   x1 <- -b / (2*a)
+   x2 <- -b / (2*a)
+   cat(sprintf("persamaan memiliki akar kembar : x1 = %.3f dan x2 = %.3f\n", x1, x2))
+ } else {
+   cat("persamaan hanya mempunyai akar-akar imajiner\n")
+ }
Error: object 'Diskriminan' not found
> |

```

Gambar 6.9 Input dan Output R Kondisi 1

```
import math

#menghitung persamaan kuadrat
# Mendefinisikan nilai setiap variabel
a = float(input("Masukkan nilai a: "))
b = float(input("Masukkan nilai b: "))
c = float(input("Masukkan nilai c: "))

# Identifikasi bentuk persamaan
if a == 0:
    print("bukan persamaan kuadrat")
else:
    Diskriminan = b**2 - 4*a*c

# pengecekan kondisi akar
if Diskriminan > 0:
    x1 = (-b + math.sqrt(Diskriminan)) / (2*a)
    x2 = (-b - math.sqrt(Diskriminan)) / (2*a)

    # Menampilkan hasil dengan format 3 desimal
    print(f"\nPersamaan memiliki akar-akar real:")
    print(f"x1 = {x1:.3f}")
    print(f"x2 = {x2:.3f}")
elif Diskriminan == 0:
    x1 = -b / (2*a)
    x2 = -b / (2*a)
    print(f"\nPersamaan memiliki akar kembar:")
    print(f"x1 = {x1:.3f}")
    print(f"x2 = {x2:.3f}")
else:
    print("\nPersamaan hanya memiliki akar-akar imajiner")
```

Hasil output di terminal Colab:
Masukkan nilai a: 0
Masukkan nilai b: 4
Masukkan nilai c: 2
bukan persamaan kuadrat

Gambar 6.10 Input dan Output Python Kondisi 1

Interpretasi output :

Pada pengujian pertama, besar nilai variabel $a = 0$, $b = 4$, dan $c = 2$, kedua program menampilkan bahwa persamaan bukan persamaan kuadrat karena program membaca pada keputusan pertama, kondisinya terpenuhi saat nilai $a = 0$. Hasil program sesuai yang diharapkan.

```
#kondisi 2
#mendefinisikan variabel
a <- as.numeric(readline(prompt = "masukkan nilai a: "))
b <- as.numeric(readline(prompt = "masukkan nilai b: "))
c <- as.numeric(readline(prompt = "masukkan nilai c: "))

#mengidentifikasi persamaan kuadrat
if (a == 0) {
  cat("persamaan bukan persamaan kuadrat\n")
} else {
  Diskriminan <- b^2 - 4*a*c
}

#mencari akar-akar persamaan
if (Diskriminan > 0) {
  x1 <- (-b + sqrt(Diskriminan)) / (2*a)
  x2 <- (-b - sqrt(Diskriminan)) / (2*a)
  cat(sprintf("akar-akar real adalah: x1 = %.3f dan x2 = %.3f\n", x1, x2))
} else if (Diskriminan == 0) {
  x1 <- -b / (2*a)
  x2 <- -b / (2*a)
  cat(sprintf("persamaan memiliki akar kembar : x1 = %.3f dan x2 = %.3f\n", x1, x2))
} else {
  cat("persamaan hanya mempunyai akar-akar imajiner\n")
}

akar-akar real adalah: x1 = 3.000 dan x2 = 2.000
```

Gambar 6.11 Input dan Output R Kondisi 2

```

import math

#menghitung persamaan kuadrat
# Mendefinisikan nilai setiap variabel
a = float(input("Masukkan nilai a: "))
b = float(input("Masukkan nilai b: "))
c = float(input("Masukkan nilai c: "))

# Identifikasi bentuk persamaan
if a == 0:
    print("bukan persamaan kuadrat")
else:
    Diskriminan = b**2 - 4*a*c

# Pengecekan kondisi akar
if Diskriminan > 0:
    x1 = (-b + math.sqrt(Diskriminan)) / (2*a)
    x2 = (-b - math.sqrt(Diskriminan)) / (2*a)

# Menampilkan hasil dengan format 3 desimal
print(f"\nPersamaan memiliki akar-akar real:")
print(f"x1 = {x1:.3f}")
print(f"x2 = {x2:.3f}")
elif Diskriminan == 0:
    x1 = -b / (2*a)
    x2 = -b / (2*a)
    print(f"\nPersamaan memiliki akar kembar:")
    print(f"x1 = {x1:.3f}")
    print(f"x2 = {x2:.3f}")
else:
    print("\nPersamaan hanya memiliki akar-akar imajiner")

```

Masukkan nilai a: 1
 Masukkan nilai b: -5
 Masukkan nilai c: 6
 Persamaan memiliki akar-akar real:
 x1 = 3.000
 x2 = 2.000

Gambar 6.12 Output Python Kondisi 2

Interpretasi output :

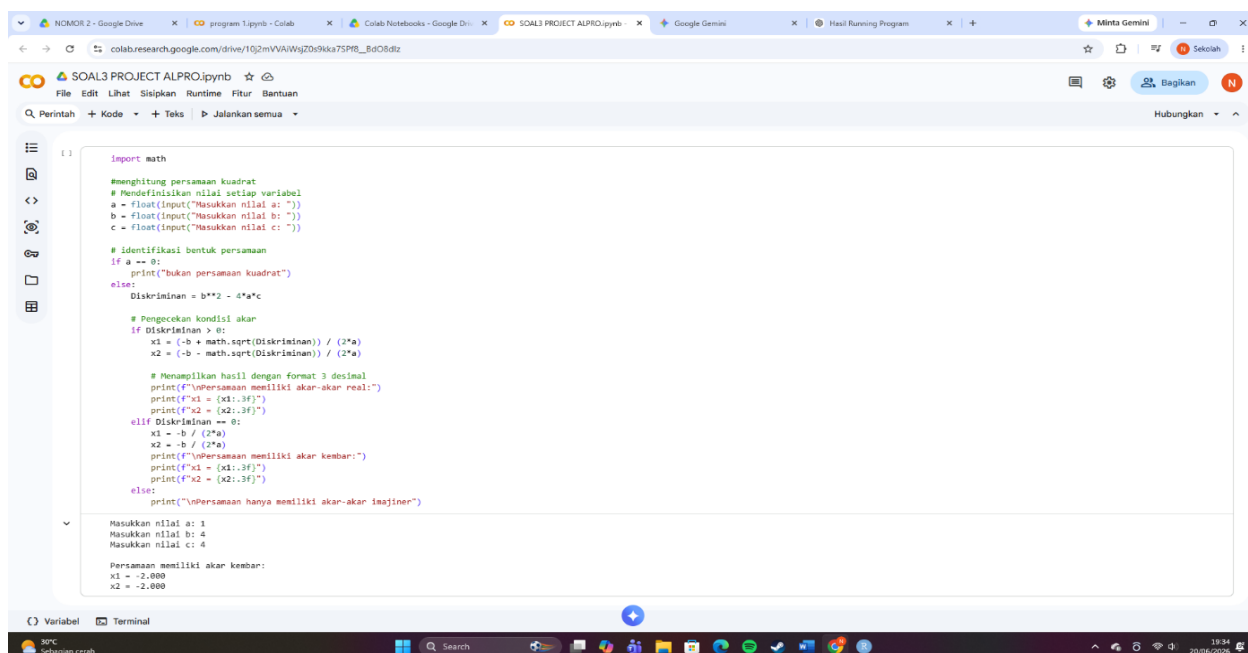
Pada pengujian kedua, dengan *input* a = 1, b = -5, dan c = 6 kedua program menampilkan bahwa persamaan memiliki nilai akar-akar real yaitu x1 = 3,000 dan x2 = 2,000. Program menghasilkan nilai tersebut karena pada keputusan pertama kondisi tidak terpenuhi sehingga program berjalan untuk melakukan perhitungan diskriminan. Setelah itu pada keputusan kedua program membaca bahwa nilai diskriminan yang diperoleh lebih dari 0 maka dilakukan perhitungan untuk mencari nilai akar-akar real.

```

#kondisi 3
> #mendefinisikan variabel
> a <- as.numeric(readline(prompt = "masukkan nilai a: "))
masukkan nilai a: 1
> b <- as.numeric(readline(prompt = "masukkan nilai b: "))
masukkan nilai b: 4
> c <- as.numeric(readline(prompt = "masukkan nilai c: "))
masukkan nilai c: 4
>
> #mengidentifikasi persamaan kuadrat
> if (a == 0) {
+   cat("persamaan bukan persamaan kuadrat\n")
+ } else {
+   Diskriminan <- b^2 - 4*a*c
+ }
>
> #mencari akar-akar persamaan
> if (Diskriminan > 0) {
+   x1 <- (-b + sqrt(Diskriminan)) / (2*a)
+   x2 <- (-b - sqrt(Diskriminan)) / (2*a)
+   cat(sprintf("akar-akar real adalah: x1 = %.3f dan x2 = %.3f\n", x1, x2))
+ } else if (Diskriminan == 0) {
+   x1 <- -b / (2*a)
+   x2 <- -b / (2*a)
+   cat(sprintf("persamaan memiliki akar kembar : x1 = %.3f dan x2 = %.3f\n", x1, x2))
+ } else {
+   cat("persamaan hanya mempunyai akar-akar imajiner\n")
+ }
>
persamaan memiliki akar kembar : x1 = -2.000 dan x2 = -2.000
>

```

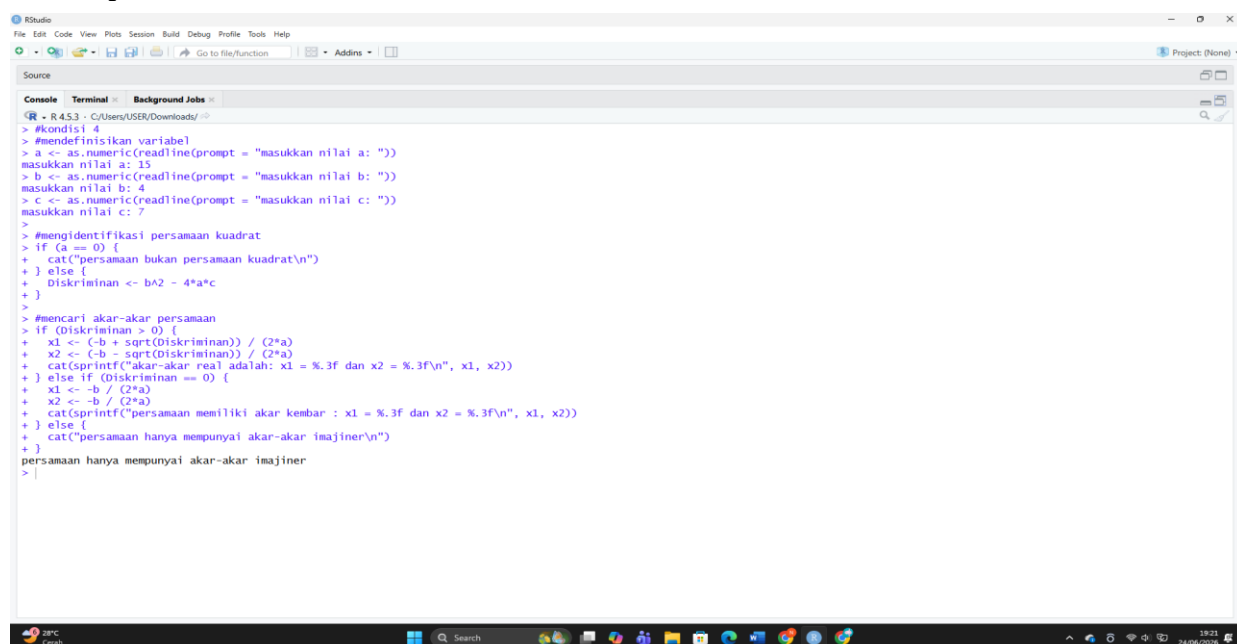
Gambar 6.13 Input dan Output R Kondisi 3



Gambar 6.14 Input dan Output Python Kondisi 3

Interpretasi output :

Pada pengujian ketiga, dengan *input* $a = 1$, $b = 4$, dan $c = 4$ kedua program menampilkan bahwa persamaan memiliki akar kembar yaitu $x_1 = -2,000$ dan $x_2 = -2,000$. Program menampilkan hasil tersebut karena pada perhitungan diskriminan diperoleh hasil diskriminan sama dengan 0 maka pada percabangan, nilai diskriminan masuk dalam kategori yang melakukan perhitungan akar kembar. Hasil sesuai dengan yang diharapkan.



Gambar 6.15 Input dan Output R Kondisi 4

```

import math

# menghitung persamaan kuadrat
# Mendefinisikan nilai setiap variabel
a = float(input("Masukkan nilai a: "))
b = float(input("Masukkan nilai b: "))
c = float(input("Masukkan nilai c: "))

# identifikasi bentuk persamaan
if a == 0:
    print("bukan persamaan kuadrat")
else:
    Diskriminan = b**2 - 4*a*c

    # pengecekan kondisi akar
    if Diskriminan > 0:
        x1 = (-b + math.sqrt(Diskriminan)) / (2*a)
        x2 = (-b - math.sqrt(Diskriminan)) / (2*a)

        # Menampilkan hasil dengan format 3 desimal
        print(f"\nPersamaan memiliki akar-akar real:")
        print(f"x1 = {x1:.3f}")
        print(f"x2 = {x2:.3f}")
    elif Diskriminan == 0:
        x1 = -b / (2*a)
        x2 = -b / (2*a)
        print(f"\nPersamaan memiliki akar kembar:")
        print(f"x1 = {x1:.3f}")
        print(f"x2 = {x2:.3f}")
    else:
        print("\nPersamaan hanya memiliki akar-akar imajiner")

```

Masukan nilai a: 14
 Masukan nilai b: 4
 Masukan nilai c: 7
 Persamaan hanya memiliki akar-akar imajiner

Gambar 6.16 Input dan Output Python Kondisi 4

Interpretasi output :

Pada pengujian keempat, dengan *input* $a = 14$, $b = 4$, dan $c = 7$ kedua program menampilkan pernyataan bahwa persamaan hanya memiliki akar imajiner. Hasil tersebut terjadi karena pada saat program melakukan perhitungan untuk menentukan nilai diskriminan, nilai yang diperoleh kurang dari 0, maka pada struktur percabangan program membaca nilai masuk pada kondisi untuk menampilkan teks tersebut. Hasil sesuai dengan yang diharapkan.

6.4 Program 4

1. Tabel pengujian

NO	Input	Output yang diharapkan	Output pada Python	Output pada R	Status	Catatan
1	NIP ASN: 123456789	Error: NIP harus terdiri dari tepat 18 digit angka	Error: NIP harus terdiri dari	Error: NIP harus terdiri dari	Berhasil	Program berjalan sesuai harapan

			tepat 18 digit angka	tepat 18 digit angka		
2	NIP ASN: 198778120934874560	Error: Bulan pada NIP tidak valid	Error: Bulan pada NIP tidak valid (harus 01-12)	Error: Bulan pada NIP tidak valid (harus 01-12)	Berhasil	Program berjalan sesuai harapan
3	NIP ASN: 204508170012436587	17 Agustus 2045	Tanggal lahir: 17 Agustus 2045	Tanggal lahir: 17 Agustus 2045	Berhasil	Program berjalan sesuai harapan

2. Hasil running

```

RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Source
Console Terminal Background Jobs
R 4.1.3 - C:/Users/USER/Downloads/...
> #kondisi 1
> nip <- readline("Masukkan NIP ASN: ")
Masukkan NIP ASN: 123456789
>
> if (nchar(nip) != 18 || grepl("[0-9]", nip)) {
+   cat("Error: NIP harus terdiri dari tepat 18 digit angka.\n")
+ } else {
+   tahun <- substr(nip, 1, 4)
+   bulan <- as.integer(substr(nip, 5, 6))
+   tanggal <- as.integer(substr(nip, 7, 8))
+
+   if (bulan < 1 || bulan > 12) {
+     cat("Error: Bulan pada NIP tidak valid (harus 01-12).\n")
+   } else {
+     if (bulan %in% c(1, 3, 5, 7, 8, 10, 12)) {
+       maks.hari <- 31
+     } else if (bulan %in% c(4, 6, 9, 11)) {
+       maks.hari <- 30
+     } else if (bulan == 2) {
+       maks.hari <- 29
+     }
+
+     if (tanggal < 1 || tanggal > maks.hari) {
+       cat(paste0("Error: Tanggal pada NIP tidak valid untuk bulan ini (harus 01-",
+         sprintf("%02d", maks.hari), ").\n"))
+     } else {
+       if (bulan == 1) {
+         nama.bulan <- "Januari"
+       } else if (bulan == 2) {
+         nama.bulan <- "Februari"
+       } else if (bulan == 3) {
+         nama.bulan <- "Maret"
+       } else if (bulan == 4) {
+         nama.bulan <- "April"
+       } else if (bulan == 5) {
+         nama.bulan <- "Mei"
+       } else if (bulan == 6) {
+         nama.bulan <- "Juni"
+       } else if (bulan == 7) {
+         nama.bulan <- "Juli"
+       } else if (bulan == 8) {
+         nama.bulan <- "Agustus"
+       }
+     }
+   }
+ }
  
```

Gambar 6.17 Input R Kondisi 1

```

+ maks_hari <- 30
+ } else if (bulan == 2) {
+   maks_hari <- 29
+ }
+
+ if (tanggal < 1 || tanggal > maks_hari) {
+   cat(paste0("Error: Tanggal pada NIP tidak valid untuk bulan ini (harus 01-",
+             sprintf("%02d", maks_hari), ").\n"))
+ } else {
+   if (bulan == 1) {
+     nama_bulan <- "Januari"
+   } else if (bulan == 2) {
+     nama_bulan <- "Februari"
+   } else if (bulan == 3) {
+     nama_bulan <- "Maret"
+   } else if (bulan == 4) {
+     nama_bulan <- "April"
+   } else if (bulan == 5) {
+     nama_bulan <- "Mei"
+   } else if (bulan == 6) {
+     nama_bulan <- "Juni"
+   } else if (bulan == 7) {
+     nama_bulan <- "Juli"
+   } else if (bulan == 8) {
+     nama_bulan <- "Agustus"
+   } else if (bulan == 9) {
+     nama_bulan <- "September"
+   } else if (bulan == 10) {
+     nama_bulan <- "Oktober"
+   } else if (bulan == 11) {
+     nama_bulan <- "November"
+   } else if (bulan == 12) {
+     nama_bulan <- "Desember"
+   }
+   cat(paste0("Tanggal lahir: ", tanggal, " ", nama_bulan, " ", tahun, "\n"))
+ }
+ }
+ }
+ Error: NIP harus terdiri dari tepat 18 digit angka.
+ >

```

Gambar 6.18 Output R Kondisi 1

```

11 # Input
12 maks_hari = 31
13 elif bulan in [4, 6, 9, 11]:
14     maks_hari = 30
15 elif bulan == 2:
16     maks_hari = 29
17
18 if tanggal < 1 or tanggal > maks_hari:
19     print(f"Error: Tanggal pada NIP tidak valid untuk bulan ini (harus 01-{maks_hari:02d}).")
20 else:
21     if bulan == 1:
22         nama_bulan = "Januari"
23     elif bulan == 2:
24         nama_bulan = "Februari"
25     elif bulan == 3:
26         nama_bulan = "Maret"
27     elif bulan == 4:
28         nama_bulan = "April"
29     elif bulan == 5:
30         nama_bulan = "Mei"
31     elif bulan == 6:
32         nama_bulan = "Juni"
33     elif bulan == 7:
34         nama_bulan = "Juli"
35     elif bulan == 8:
36         nama_bulan = "Agustus"
37     elif bulan == 9:
38         nama_bulan = "September"
39     elif bulan == 10:
40         nama_bulan = "Oktober"
41     elif bulan == 11:
42         nama_bulan = "November"
43     elif bulan == 12:
44         nama_bulan = "Desember"
45
46     print(f"Tanggal lahir: {tanggal} {nama_bulan} {tahun}")

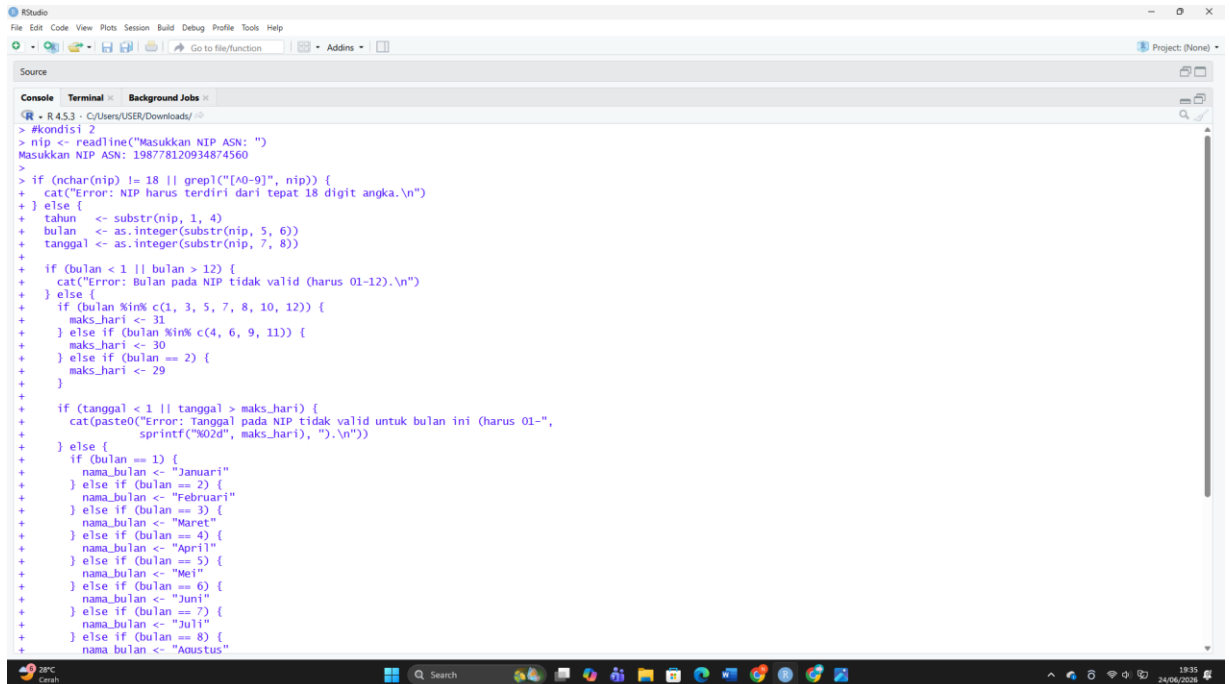
```

... Masukkan NIP ASH: 123456789
Error: NIP harus terdiri dari tepat 18 digit angka.

Gambar 6.19 Input dan Output Python Kondisi 1

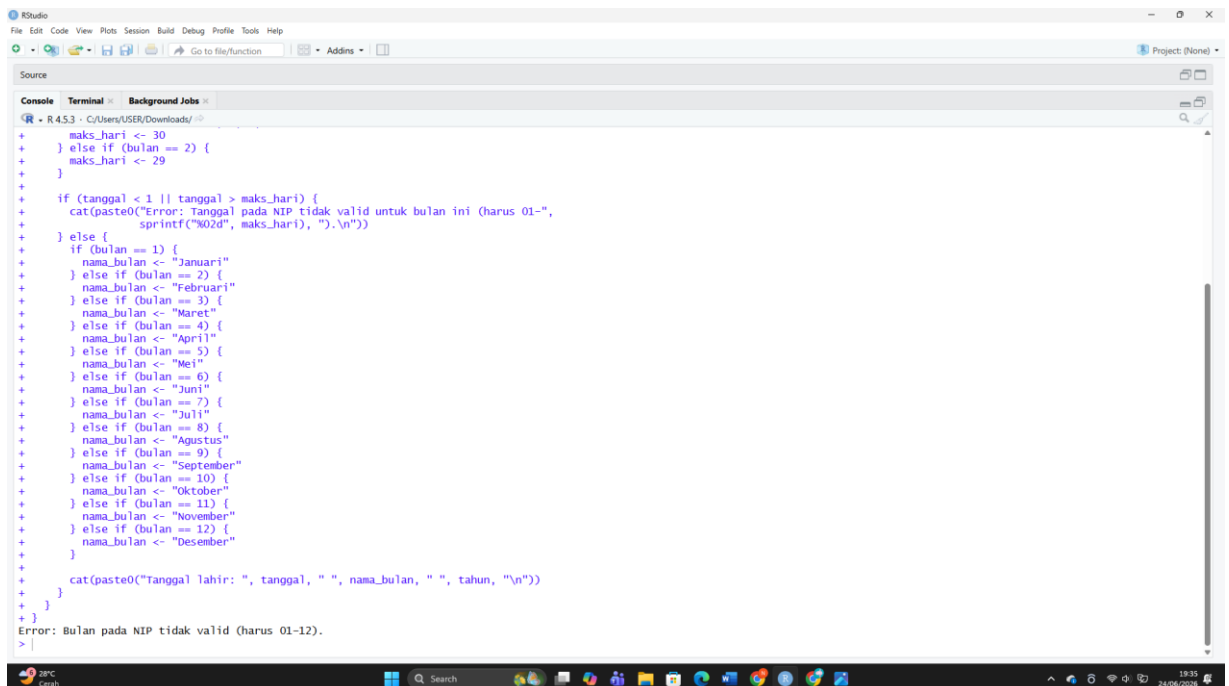
Interpretasi output :

Pada pengujian pertama, menginput NIP yaitu 123456789. Program langsung menyadari bahwa jumlah angka tersebut baru 9 digit, tidak sesuai batas yang mana seharusnya NIP ASN memiliki 18 digit angka, oleh karna itu program menjalankan dan outputnya “Error:NIP harus terdiri dari tepat 18 digit angka”.



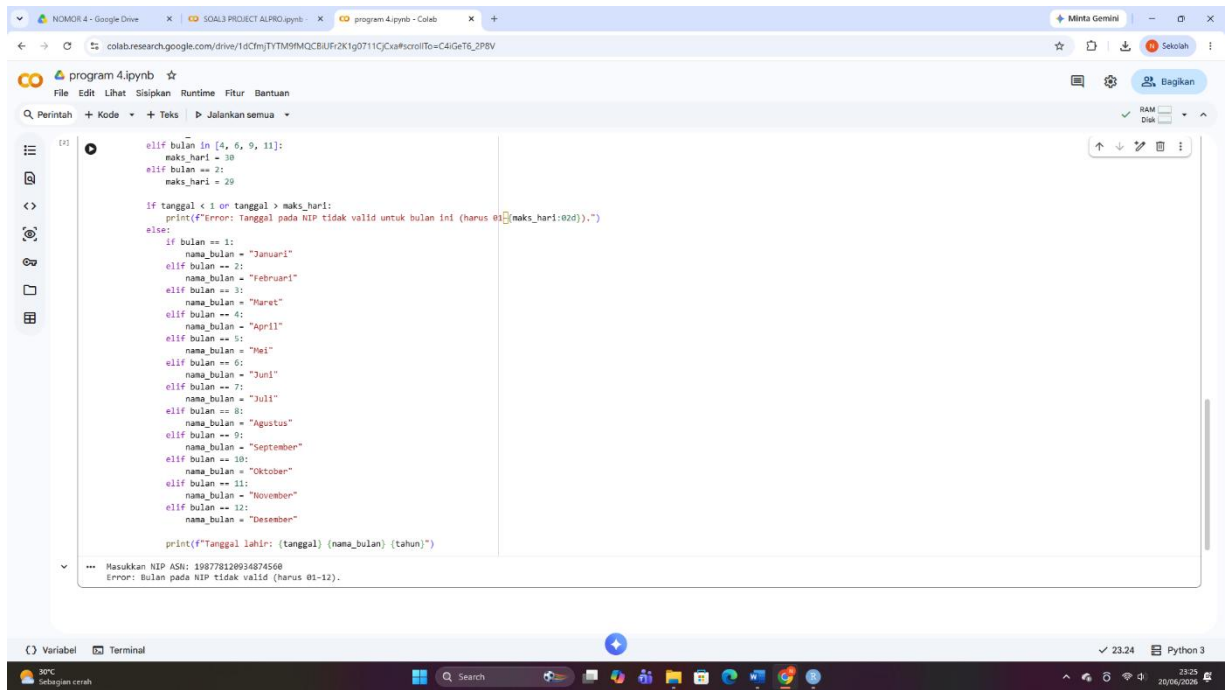
```
RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Source
Console Terminal Background Jobs
R 4.5.3 - C:/Users/USER/Downloads/
> #kondisi 2
> nip <- readline("Masukkan NIP ASN: ")
Masukkan NIP ASN: 198778120934874560
>
> if (nchar(nip) != 18 || grepl("[0-9]", nip)) {
+   cat("Error: NIP harus terdiri dari tepat 18 digit angka.\n")
+ } else {
+   tahun <- substr(nip, 1, 4)
+   bulan <- as.integer(substr(nip, 5, 6))
+   tanggal <- as.integer(substr(nip, 7, 8))
+
+   if (bulan < 1 || bulan > 12) {
+     cat("Error: Bulan pada NIP tidak valid (harus 01-12).\n")
+   } else {
+     if (bulan %in% c(1, 3, 5, 7, 8, 10, 12)) {
+       maks_hari <- 31
+     } else if (bulan %in% c(4, 6, 9, 11)) {
+       maks_hari <- 30
+     } else if (bulan == 2) {
+       maks_hari <- 29
+     }
+
+     if (tanggal < 1 || tanggal > maks_hari) {
+       cat(paste0("Error: Tanggal pada NIP tidak valid untuk bulan ini (harus 01-",
+         sprintf("%02d", maks_hari), ").\n"))
+     } else {
+       if (bulan == 1) {
+         nama_bulan <- "Januari"
+       } else if (bulan == 2) {
+         nama_bulan <- "Februari"
+       } else if (bulan == 3) {
+         nama_bulan <- "Maret"
+       } else if (bulan == 4) {
+         nama_bulan <- "April"
+       } else if (bulan == 5) {
+         nama_bulan <- "Mei"
+       } else if (bulan == 6) {
+         nama_bulan <- "Juni"
+       } else if (bulan == 7) {
+         nama_bulan <- "Juli"
+       } else if (bulan == 8) {
+         nama_bulan <- "Agustus"
+       }
+     }
+   }
+ }
+ cat(paste0("Tanggal lahir: ", tanggal, " ", nama_bulan, " ", tahun, "\n"))
+ }
```

Gambar 6.20 Input R Kondisi 2



```
RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Source
Console Terminal Background Jobs
R 4.5.3 - C:/Users/USER/Downloads/
+ maks_hari <- 30
+ } else if (bulan == 2) {
+   maks_hari <- 29
+ }
+
+ if (tanggal < 1 || tanggal > maks_hari) {
+   cat(paste0("Error: Tanggal pada NIP tidak valid untuk bulan ini (harus 01-",
+     sprintf("%02d", maks_hari), ").\n"))
+ } else {
+   if (bulan == 1) {
+     nama_bulan <- "Januari"
+   } else if (bulan == 2) {
+     nama_bulan <- "Februari"
+   } else if (bulan == 3) {
+     nama_bulan <- "Maret"
+   } else if (bulan == 4) {
+     nama_bulan <- "April"
+   } else if (bulan == 5) {
+     nama_bulan <- "Mei"
+   } else if (bulan == 6) {
+     nama_bulan <- "Juni"
+   } else if (bulan == 7) {
+     nama_bulan <- "Juli"
+   } else if (bulan == 8) {
+     nama_bulan <- "Agustus"
+   } else if (bulan == 9) {
+     nama_bulan <- "September"
+   } else if (bulan == 10) {
+     nama_bulan <- "Oktober"
+   } else if (bulan == 11) {
+     nama_bulan <- "November"
+   } else if (bulan == 12) {
+     nama_bulan <- "Desember"
+   }
+ }
+ cat(paste0("Tanggal lahir: ", tanggal, " ", nama_bulan, " ", tahun, "\n"))
+ }
+ }
+ }
Error: Bulan pada NIP tidak valid (harus 01-12).
>
```

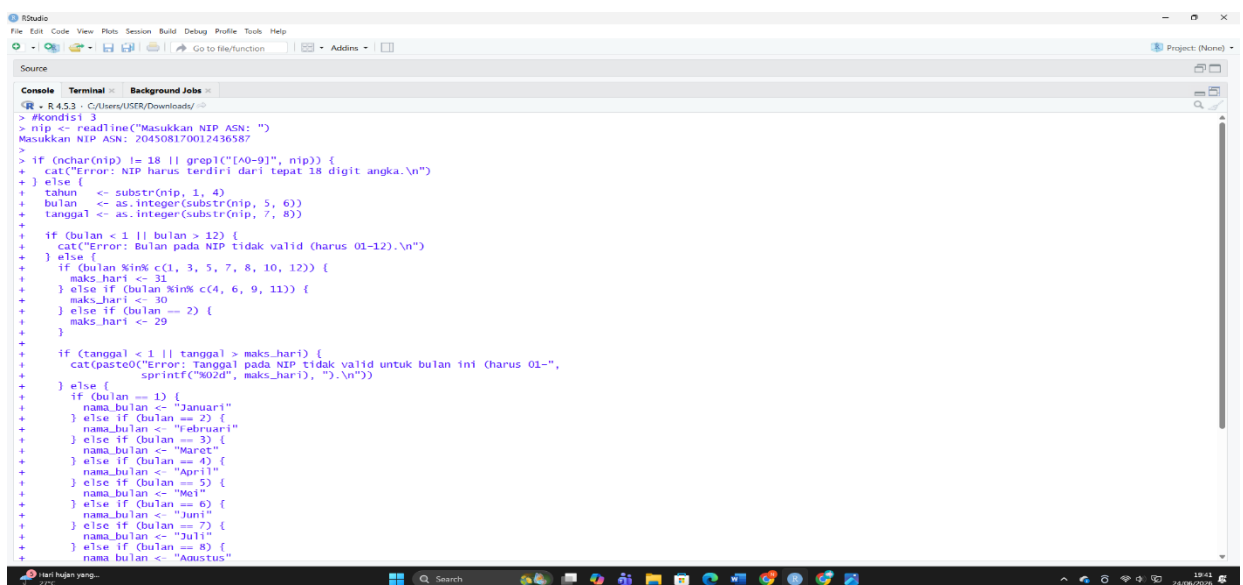
Gambar 6.21 Output R Kondisi 2



Gambar 6.22 Input dan Output Python Kondisi 2

Interpretasi output :

Pada pengujian kedua, menginput NIP yaitu 198778120934874560. Program menunjukkan NIP tepat 18 digit angka akan tetapi, program mencoba membedah delapan digit pertamanya (19877812), sistem mendeteksi kegagalan logika. Angka tersebut mengisyaratkan kelahiran pada tahun 1987, tanggal 12, di bulan 78. karena tidak ada bulan ke 78 maka, program kembali menghentikan jalanya program dan outputnya “Error:Bulan pada NIP tidak valid(harus 01-12)”.



Gambar 6.23 Input R Kondisi 3

```

RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Source
Console Terminal Background Jobs
R 4.5.3 - C:/Users/USER/Downloads/
+ maks_hari <- 30
+ } else if (bulan == 2) {
+   maks_hari <- 29
+ }
+
+ if (tanggal < 1 || tanggal > maks_hari) {
+   cat(paste0("Error: Tanggal pada NIP tidak valid untuk bulan ini (harus 01-",
+             sprintf("%02d", maks_hari), ").\n"))
+ } else {
+   if (bulan == 1) {
+     nama_bulan <- "Januari"
+   } else if (bulan == 2) {
+     nama_bulan <- "Februari"
+   } else if (bulan == 3) {
+     nama_bulan <- "Maret"
+   } else if (bulan == 4) {
+     nama_bulan <- "April"
+   } else if (bulan == 5) {
+     nama_bulan <- "Mei"
+   } else if (bulan == 6) {
+     nama_bulan <- "Juni"
+   } else if (bulan == 7) {
+     nama_bulan <- "Juli"
+   } else if (bulan == 8) {
+     nama_bulan <- "Agustus"
+   } else if (bulan == 9) {
+     nama_bulan <- "September"
+   } else if (bulan == 10) {
+     nama_bulan <- "Oktober"
+   } else if (bulan == 11) {
+     nama_bulan <- "November"
+   } else if (bulan == 12) {
+     nama_bulan <- "Desember"
+   }
+   cat(paste0("Tanggal lahir: ", tanggal, " ", nama_bulan, " ", tahun, "\n"))
+ }
+ }
+
Tanggal lahir: 17 Agustus 2045
>

```

Gambar 6.24 Output R Kondisi 3

```

Google Drive X SOAL3 PROJECT ALPRO.ipynb X program 4.py - Colab X
colabresearch.google.com/drive/1dCfmjTYTM9MQCBUR2K1g0711CjCxa#scrollTo=C4GeT6_2PBV
program 4.py
File Edit Lihat Sisipkan Runtime Fitur Bantuan
Perintah Kode Teks Jalankan semua
[1] 4/
elif bulan in [4, 6, 9, 11]:
    maks_hari = 30
elif bulan == 2:
    maks_hari = 29

if tanggal < 1 or tanggal > maks_hari:
    print(f"Error: Tanggal pada NIP tidak valid untuk bulan ini (harus 01-{maks_hari:02d}).")
else:
    if bulan == 1:
        nama_bulan = "Januari"
    elif bulan == 2:
        nama_bulan = "Februari"
    elif bulan == 3:
        nama_bulan = "Maret"
    elif bulan == 4:
        nama_bulan = "April"
    elif bulan == 5:
        nama_bulan = "Mei"
    elif bulan == 6:
        nama_bulan = "Juni"
    elif bulan == 7:
        nama_bulan = "Juli"
    elif bulan == 8:
        nama_bulan = "Agustus"
    elif bulan == 9:
        nama_bulan = "September"
    elif bulan == 10:
        nama_bulan = "Oktober"
    elif bulan == 11:
        nama_bulan = "November"
    elif bulan == 12:
        nama_bulan = "Desember"

    print(f"Tanggal lahir: {tanggal} {nama_bulan} {tahun}")

--- Hasilkan NIP ASK: 204508170012436587
    Tanggal lahir: 17 Agustus 2045

```

Gambar 6.25 Input dan Output Python Kondisi 3

Interpretasi output :

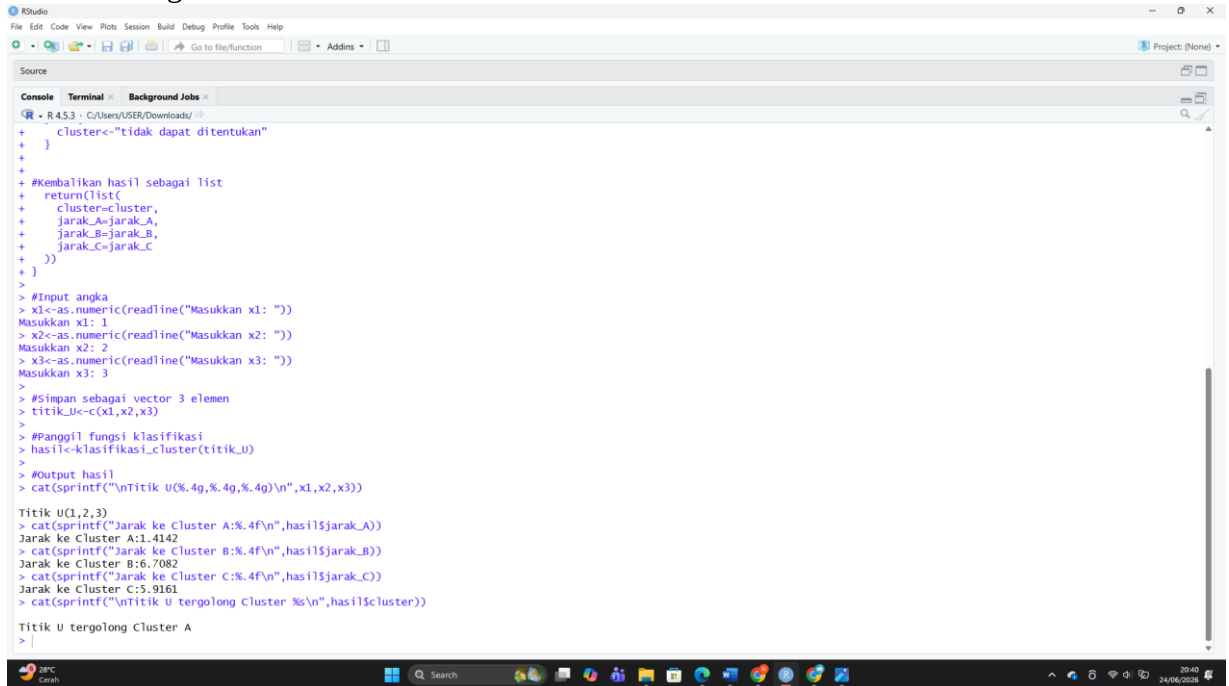
Pada pengujian ketiga, menginput NIP yaitu 204508170012436587. Program menunjukkan NIP tepat 18 digit angka dan 8 digit angkanya sesuai dengan identitas atau terbaca menjadi tanggal lahir identitas, sehingga outputnya “Tanggal lahir:17 agustus 2045”.

6.5 Program 5

1. Tabel pengujian

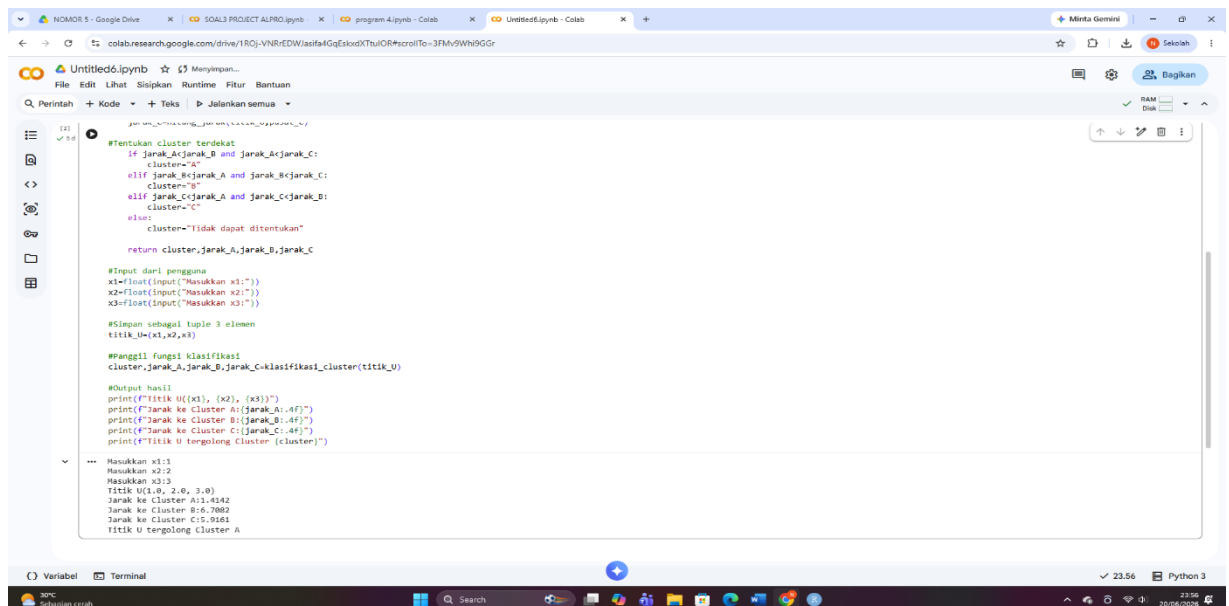
No	Input	Output yang diharapkan	Output pada Python	Output pada R	Status	Catatan
1	X1: 1 X2: 2 X3: 3	Jarak A:1.4142 B:6.7082 C:5.9161 Titik U tergolong Cluster A	Jarak A:1.4142 B:6.7082 C:5.9161 Titik U tergolong Cluster A	Jarak A:1.4142 B:6.7082 C:5.9161 Titik U tergolong Cluster A	Berhasil	Program berjalan sesuai harapan
2	X1: -1 X2: 5 X3: -2	Jarak A:7.0711 B:12.2066 C:2.2361 Titik U tergolong Cluster C	Jarak A:7.0711 B:12.2066 C:2.2361 Titik U tergolong Cluster C	Jarak A:7.0711 B:12.2066 C:2.2361 Titik U tergolong Cluster C	Berhasil	Program berjalan sesuai harapan
3	X1: -10 X2: -8 X3: -1	Jarak A:15.5242 B:13.6382 C:13.6382 Titik U tergolong tidak dapat ditentukan	Jarak A:15.5242 B:13.6382 C:13.6382 Titik U tergolong tidak dapat ditentukan	Jarak A:15.5242 B:13.6382 C:13.6382 Titik U tergolong tidak dapat ditentukan	Berhasil	Program berjalan sesuai harapan

2. Hasil running



```
Source
Console Terminal Background Jobs
R 4.5.3 - C:/Users/USER/Downloads/...
+ cluster<-"tidak dapat ditentukan"
+ }
+
+ #Kembalikan hasil sebagai list
+ return(list(
+   cluster=cluster,
+   jarak_A=jarak_A,
+   jarak_B=jarak_B,
+   jarak_C=jarak_C
+ ))
+ }
+
> #Input angka
> x1<-as.numeric(readline("Masukkan x1: "))
Masukkan x1: 1
> x2<-as.numeric(readline("Masukkan x2: "))
Masukkan x2: 2
> x3<-as.numeric(readline("Masukkan x3: "))
Masukkan x3: 3
>
> #Simpan sebagai vector 3 elemen
> titik_U<-c(x1,x2,x3)
>
> #Panggil fungsi klasifikasi
> hasil<-klasifikasi_cluster(titik_U)
>
> #Output hasil
> cat(sprintf("\nTitik U(%g,%g,%g)\n",x1,x2,x3))
Titik U(1,2,3)
> cat(sprintf("Jarak ke Cluster A:%.4f\n",hasil$jarak_A))
Jarak ke Cluster A:1.4142
> cat(sprintf("Jarak ke Cluster B:%.4f\n",hasil$jarak_B))
Jarak ke Cluster B:6.7082
> cat(sprintf("Jarak ke Cluster C:%.4f\n",hasil$jarak_C))
Jarak ke Cluster C:5.9161
> cat(sprintf("\nTitik U tergolong Cluster %s\n",hasil$cluster))
Titik U tergolong Cluster A
>
```

Gambar 6.26 Input dan Output R Kondisi 1



```
Untitled6.ipynb
File Edit Lihat Simpan Runtime Fitur Bantuan
Perintah Kode Teks Jalankan semua
#tentukan Cluster terdekat
if jarak_A<jarak_B and jarak_A<jarak_C:
    cluster="A"
elif jarak_B<jarak_A and jarak_B<jarak_C:
    cluster="B"
elif jarak_C<jarak_A and jarak_C<jarak_B:
    cluster="C"
else:
    cluster="Tidak dapat ditentukan"
return cluster, jarak_A, jarak_B, jarak_C

#Input dari pengguna
x1=float(input("Masukkan x1:"))
x2=float(input("Masukkan x2:"))
x3=float(input("Masukkan x3:"))

#Simpan sebagai tuple 3 elemen
titik_U=(x1,x2,x3)

#Panggil fungsi klasifikasi
cluster, jarak_A, jarak_B, jarak_C=klasifikasi_cluster(titik_U)

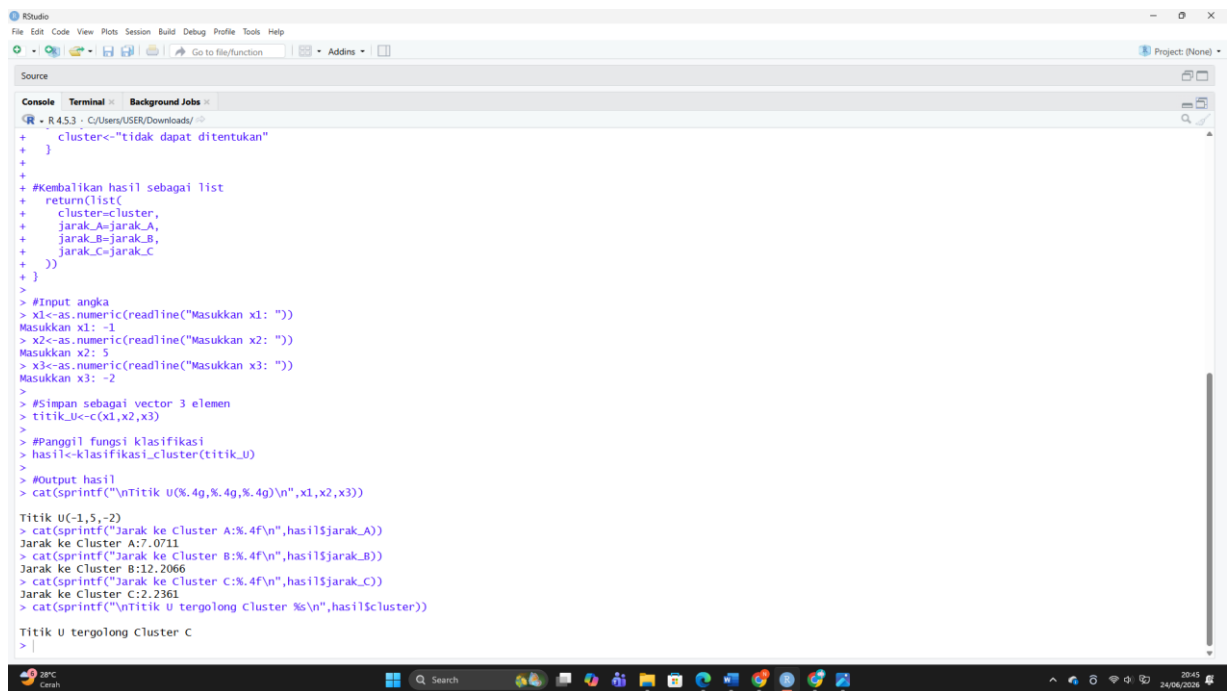
#Output hasil
print(f"Titik U({x1}, {x2}, {x3})")
print(f"Jarak ke Cluster A:{jarak_A:.4f}")
print(f"Jarak ke Cluster B:{jarak_B:.4f}")
print(f"Jarak ke Cluster C:{jarak_C:.4f}")
print(f"Titik U tergolong Cluster ({cluster})")

Masukkan x1:1
Masukkan x2:2
Masukkan x3:3
Titik U(1.0, 2.0, 3.0)
Jarak ke Cluster A:1.4142
Jarak ke Cluster B:6.7082
Jarak ke Cluster C:5.9161
Titik U tergolong Cluster A
```

Gambar 6.27 Input dan Output Python Kondisi 1

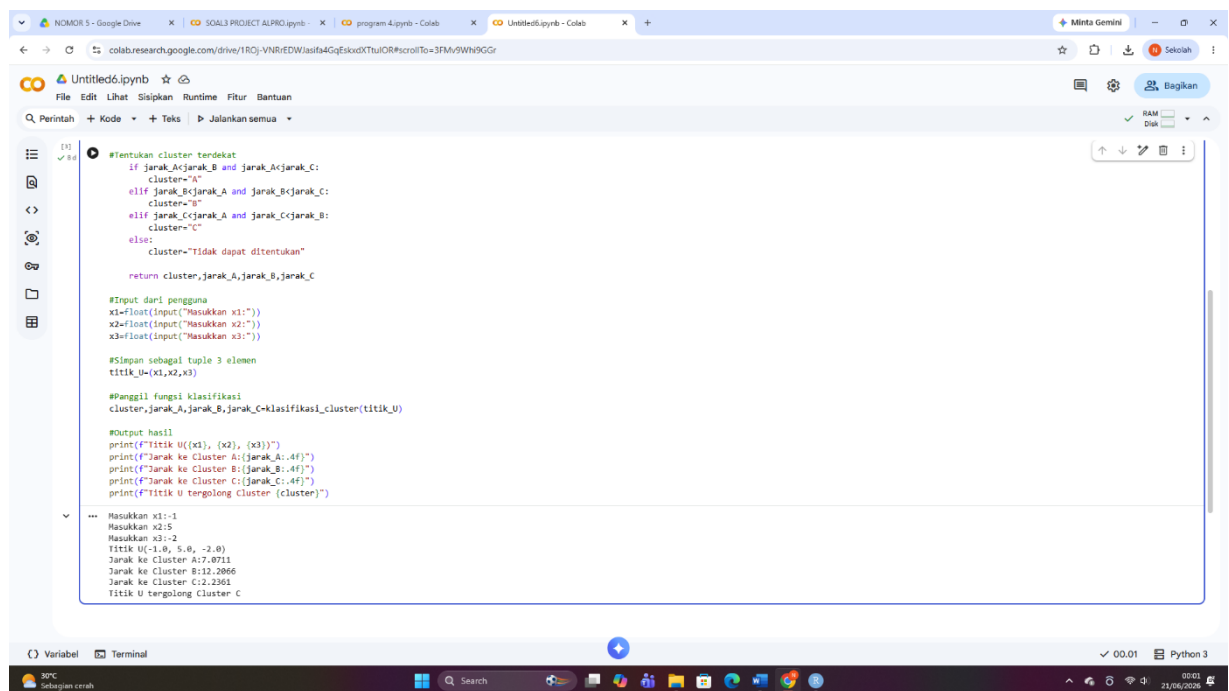
Interpretasi output :

Pada pengujian pertama dengan input titik U(1,2,3) diperoleh hasil jarak ke Cluster A sebesar 1.4142, Cluster B sebesar 6.7082, dan Cluster C sebesar 5.9161. Dari hasil tersebut, jarak terkecil berada pada Cluster A, sehingga titik U diklasifikasikan ke dalam Cluster A. Hal ini menunjukkan bahwa program mampu menentukan cluster berdasarkan kedekatan titik terhadap pusat cluster.



```
Source
Console Terminal Background Jobs
R - R 4.5.3 - C:/Users/USER/Downloads/
cluster<- "tidak dapat ditentukan"
}
}
#Kembalikan hasil sebagai list
return(list(
cluster=cluster,
jarak_A=jarak_A,
jarak_B=jarak_B,
jarak_C=jarak_C
))
}
}
> #Input angka
> x1<-as.numeric(readline("Masukkan x1: "))
Masukkan x1: -1
> x2<-as.numeric(readline("Masukkan x2: "))
Masukkan x2: 5
> x3<-as.numeric(readline("Masukkan x3: "))
Masukkan x3: -2
>
> #Simpan sebagai vector 3 elemen
> titik_U<-c(x1,x2,x3)
>
> #Panggil fungsi klasifikasi
> hasil<-klasifikasi_cluster(titik_U)
>
> #Output hasil
> cat(sprintf("\nTitik U(%.4g,%.4g,%.4g)\n",x1,x2,x3))
Titik U(-1,5,-2)
> cat(sprintf("Jarak ke Cluster A:%.4f\n",hasil$jarak_A))
Jarak ke Cluster A:7.0711
> cat(sprintf("Jarak ke Cluster B:%.4f\n",hasil$jarak_B))
Jarak ke Cluster B:12.2066
> cat(sprintf("Jarak ke Cluster C:%.4f\n",hasil$jarak_C))
Jarak ke Cluster C:2.2361
> cat(sprintf("\nTitik U tergolong Cluster %s\n",hasil$cluster))
Titik U tergolong Cluster C
>
```

Gambar 6.28 Input dan Output R Kondisi 2



```
Untitled6.ipynb
File Edit Lihat Simpan Runtime Fitur Bantuan
Perintah + Kode + Teks ▶ Jalankan semua
#Tentukan cluster terdekat
if jarak_A<jarak_B and jarak_A<jarak_C:
    cluster="A"
elif jarak_B<jarak_A and jarak_B<jarak_C:
    cluster="B"
elif jarak_C<jarak_A and jarak_C<jarak_B:
    cluster="C"
else:
    cluster="Tidak dapat ditentukan"
return cluster, jarak_A, jarak_B, jarak_C

#Input dari pengguna
x1=float(input("Masukkan x1:"))
x2=float(input("Masukkan x2:"))
x3=float(input("Masukkan x3:"))

#Simpan sebagai tuple 3 elemen
titik_U=(x1,x2,x3)

#Panggil fungsi klasifikasi
cluster, jarak_A, jarak_B, jarak_C=klasifikasi_cluster(titik_U)

#Output hasil
print(f"Titik U({x1}, {x2}, {x3})")
print(f"Jarak ke Cluster A:{jarak_A:.4f}")
print(f"Jarak ke Cluster B:{jarak_B:.4f}")
print(f"Jarak ke Cluster C:{jarak_C:.4f}")
print(f"Titik U tergolong Cluster {cluster}")

Masukkan x1:-1
Masukkan x2:5
Masukkan x3:-2
Titik U(-1.0, 5.0, -2.0)
Jarak ke Cluster A:7.0711
Jarak ke Cluster B:12.2066
Jarak ke Cluster C:2.2361
Titik U tergolong Cluster C
```

Gambar 6.29 Input dan Output Python Kondisi 2

Interpretasi output :

ada pengujian kedua dengan input titik U(-1,5,-2) diperoleh jarak ke Cluster A sebesar 7.0711, Cluster B sebesar 12.2066, dan Cluster C sebesar 2.2361. Karena jarak paling kecil terdapat pada Cluster C, maka titik tersebut termasuk ke dalam Cluster C. Hasil ini

membuktikan bahwa program dapat menangani input dengan koordinat negatif dan tetap menghasilkan klasifikasi yang tepat.

```

Source
Console Terminal Background Jobs
R - R 4.5.3 - C:/Users/USER/Downloads/
+ cluster<-"tidak dapat ditentukan"
+ }
+
+ #Kembalikan hasil sebagai list
+ return(list(
+   cluster=cluster,
+   jarak_A=jarak_A,
+   jarak_B=jarak_B,
+   jarak_C=jarak_C
+ ))
+ )
+
> #Input angka
> x1<-as.numeric(readline("Masukkan x1: "))
Masukkan x1: -10
> x2<-as.numeric(readline("Masukkan x2: "))
Masukkan x2: -8
> x3<-as.numeric(readline("Masukkan x3: "))
Masukkan x3: -1
>
> #Simpan sebagai vector 3 elemen
> titik_U<-c(x1,x2,x3)
>
> #Panggil fungsi klasifikasi
> hasil<-klasifikasi_cluster(titik_U)
>
> #Output hasil
> cat(sprintf("\nTitik U(%g,%g,%g)\n",x1,x2,x3))
Titik U(-10,-8,-1)
> cat(sprintf("Jarak ke Cluster A:%.4f\n",hasil$jarak_A))
Jarak ke Cluster A:15.5242
> cat(sprintf("Jarak ke Cluster B:%.4f\n",hasil$jarak_B))
Jarak ke Cluster B:13.6382
> cat(sprintf("Jarak ke Cluster C:%.4f\n",hasil$jarak_C))
Jarak ke Cluster C:13.6382
> cat(sprintf("\nTitik U tergolong Cluster %s\n",hasil$cluster))
Titik U tergolong Cluster tidak dapat ditentukan
>

```

Gambar 6.30 Input dan Output R Kondisi 3

```

Untitled6.ipynb
File Edit Lihat Sisipkan Runtime Fitur Bantuan
Perintah Kode Teks Jalankan semua
[4]
#tentukan cluster terdekat
if jarak_A<jarak_B and jarak_A<jarak_C:
    cluster="A"
elif jarak_B<jarak_A and jarak_B<jarak_C:
    cluster="B"
elif jarak_C<jarak_A and jarak_C<jarak_B:
    cluster="C"
else:
    cluster="Tidak dapat ditentukan"
return cluster,jarak_A,jarak_B,jarak_C

#Input dari pengguna
x1=float(input("Masukkan x1:"))
x2=float(input("Masukkan x2:"))
x3=float(input("Masukkan x3:"))

#Simpan sebagai tuple 3 elemen
titik_U=(x1,x2,x3)

#Panggil fungsi klasifikasi
cluster,jarak_A,jarak_B,jarak_C=klasifikasi_cluster(titik_U)

#Output hasil
print(f"Titik U({x1}, {x2}, {x3})")
print(f"Jarak ke Cluster A:{jarak_A:.4f}")
print(f"Jarak ke Cluster B:{jarak_B:.4f}")
print(f"Jarak ke Cluster C:{jarak_C:.4f}")
print(f"Titik U tergolong Cluster {cluster}")

Masukkan x1:-10
Masukkan x2:-8
Masukkan x3:-1
Titik U(-10,-8,-1)
Jarak ke Cluster A:15.5242
Jarak ke Cluster B:13.6382
Jarak ke Cluster C:13.6382
Titik U tergolong Cluster Tidak dapat ditentukan

```

Gambar 6.31 Input dan Output Python Kondisi 3

Interpretasi output :

Pada pengujian ketiga dengan input titik U(-10,-8,-1) diperoleh jarak ke Cluster A sebesar 15.5242, Cluster B sebesar 13.6382, dan Cluster C sebesar 13.6382. Karena

Cluster B dan Cluster C memiliki jarak yang sama, maka program memberikan hasil tidak dapat ditentukan. Kondisi ini sesuai dengan aturan program bahwa jika terdapat dua cluster dengan jarak sama, maka titik dianggap berada pada batas antar cluster.

6.6 Program 6

1. Tabel pengujian

No	Input	Output yang diharapkan	Output paada Python	Output pada R	Status	Catatan
1	p_hat = 0.567 n = 10 alpha = 0.05	Batas Bawah = 0.2598 Batas Atas = 0.8741 Interval konfidensi = (0.2599, 0.8741)	Batas Bawah = 0.25989170353114843 Batas Atas = 0.8741082964688515 Interval konfidensi = (0.2599 , 0.8741)	Batas bawah = 0.2598917 Batas atas = 0.8741083 Interval konfidensi = (0.2599 , 0.8741)	Berhasil	Program berjalan sesuai harapan
2	p_hat = 1.5 n = 10 alpha = 0.05	Error: Proporsi sampel harus berada pada rentang 0 sampai 1	Error: Proporsi sampel harus berada pada rentang 0 sampai 1	Error: p_hat harus berada antara 0 dan 1	Berhasil	Program berjalan sesuai harapan
3	p_hat = 0.567 n = -10 alpha = 0.05	Error: Ukuran sampel harus lebih dari 0	Error: Ukuran sampel harus lebih dari 0	Error: Ukuran sampel harus lebih dari 0	Berhasil	Program berjalan sesuai harapan
4	p_hat = 0.567 n = 10	Error: Alpha harus	Error: Alpha harus bernilai 0.05 atau 0.10	Error: alpha harus 0.05 atau 0.10	Berhasil	Program berjalan

	alpha = 0.01	bernilai 0.05 atau 0.10				sesuai harapan
--	-----------------	----------------------------	--	--	--	-------------------

2. Hasil running

```

+ z <- 1.96
+ } else if (alpha == 0.10) {
+   z <- 1.645
+ }
+ margin_error <- z * sqrt(p_hat * (1 - p_hat)) / n
+ batas_bawah <- p_hat - margin_error
+ batas_atas <- p_hat + margin_error
+ return(list(
+   BB = batas_bawah,
+   BA = batas_atas
+ ))
+ }
+ }
+ # Input
+ p_hat <- 0.567
+ n <- 10
+ alpha <- 0.05
+ # Validasi
+ if (p_hat < 0 || p_hat > 1) {
+   cat("Error: p_hat harus berada antara 0 dan 1\n")
+ } else if (n <= 0) {
+   cat("Error: ukuran sampel harus lebih dari 0\n")
+ } else if (alpha != 0.05 && alpha != 0.10) {
+   cat("Error: alpha harus 0.05 atau 0.10\n")
+ } else {
+   hasil <- hitung_interval(p_hat, n, alpha)
+   # Menampilkan Hasil
+   cat("Batas bawah =", hasil$BB, "\n")
+   cat("Batas atas =", hasil$BA, "\n")
+   cat("Interval konfidensi = (", round(hasil$BB, 4), ", ", round(hasil$BA, 4), ")\n")
+ }
+
Batas bawah = 0.2598917
Batas atas = 0.8741083
Interval konfidensi = ( 0.2599 , 0.8741 )
>

```

Gambar 6.32 Output R Kondisi 1

```

# Input
p_hat = 0.567
n = 10
alpha = 0.05

# Validasi
if p_hat < 0 or p_hat > 1:
    print("Error: Proporsi sampel harus berada pada rentang 0 sampai 1")

elif n <= 0:
    print("Error: Ukuran sampel harus lebih dari 0")

elif alpha != 0.05 and alpha != 0.10:
    print("Error: Alpha harus bernilai 0.05 atau 0.10")

else:
    hasil = hitung_interval(p_hat, n, alpha)

# Menampilkan Hasil
print("Batas Bawah =", hasil["BB"])
print("Batas Atas =", hasil["BA"])
print("Interval konfidensi = (", round(hasil["BB"], 4), ", ", round(hasil["BA"], 4), ")")

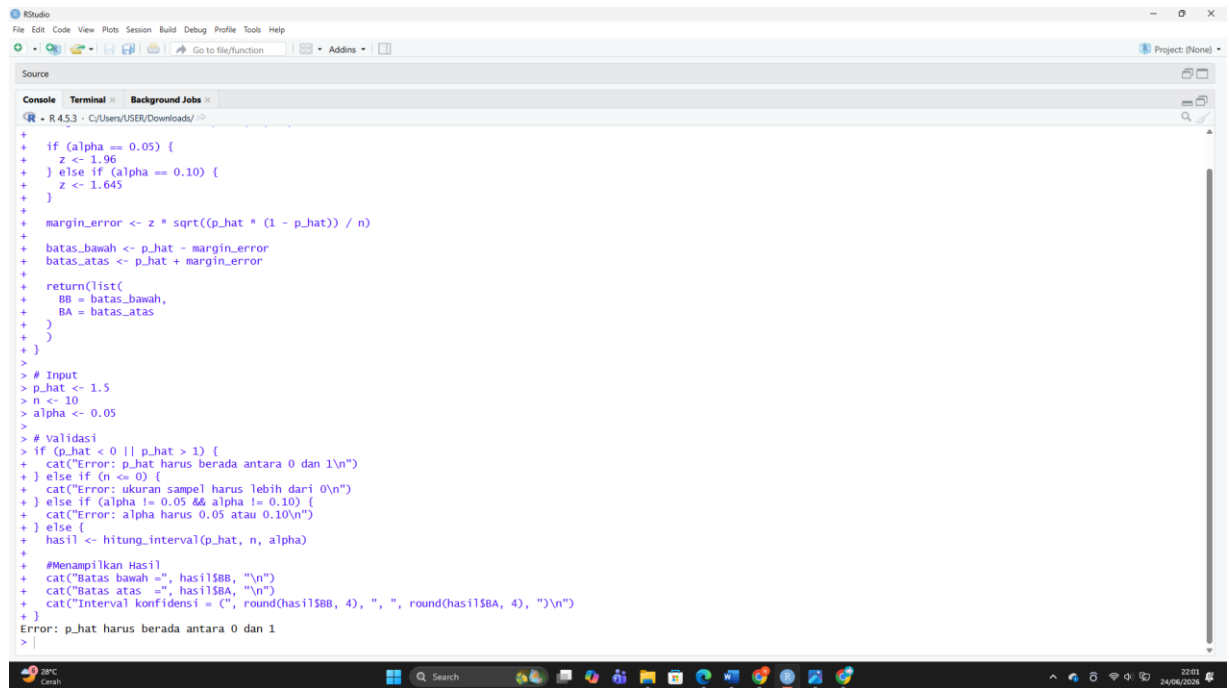
...
Batas Bawah = 0.25989170353114843
Batas Atas = 0.8741082964688515
Interval konfidensi = ( 0.2599 , 0.8741 )

```

Gambar 6.33 Output Python Kondisi 1

Interpretasi Output :

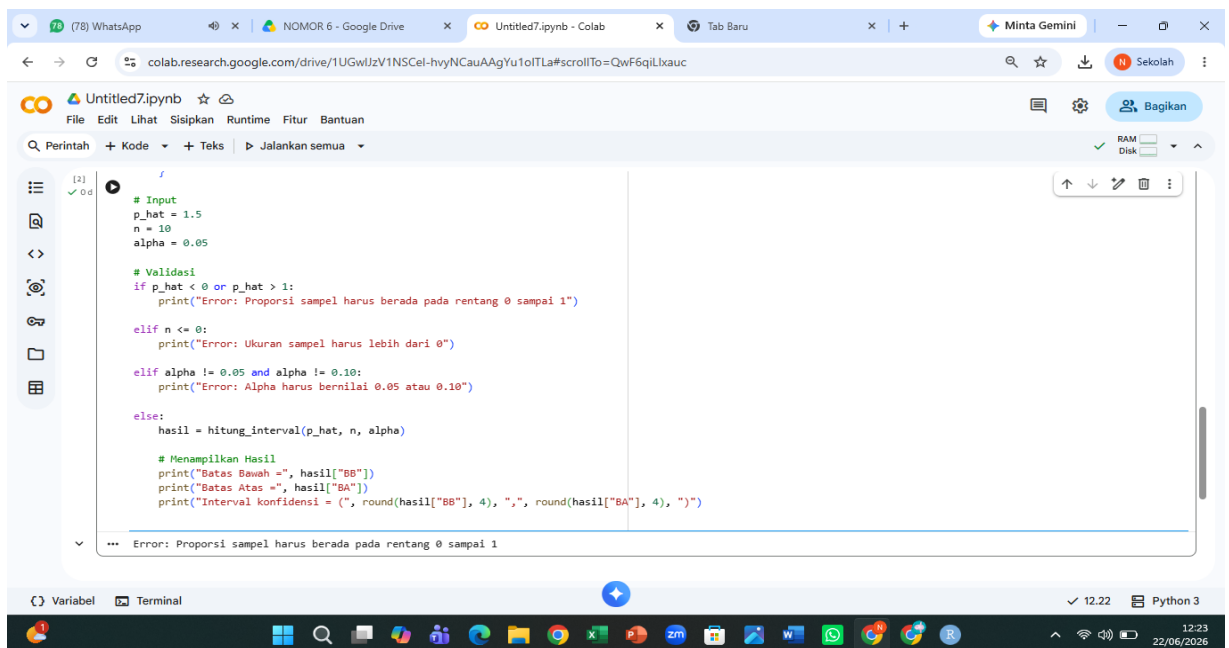
Pada pengujian pertama, digunakan input $p_hat = 0,567$, $n = 10$, dan $\alpha = 0,05$. Program berhasil menghitung interval konfidensi proporsi dengan batas bawah sekitar 0,2599 dan batas atas sekitar 0,8741. Hasil yang diperoleh pada Python dan R memiliki nilai yang sama, meskipun terdapat perbedaan jumlah digit desimal yang ditampilkan. Hal ini menunjukkan bahwa proses perhitungan interval konfidensi telah berjalan dengan benar pada kedua bahasa pemrograman.



```
Source
Console Terminal Background Jobs
R - R 4.5.3 - C:/Users/USER/Downloads/

+ if (alpha == 0.05) {
+   z <- 1.96
+ } else if (alpha == 0.10) {
+   z <- 1.645
+ }
+ margin_error <- z * sqrt((p_hat * (1 - p_hat)) / n)
+ batas_bawah <- p_hat - margin_error
+ batas_atas <- p_hat + margin_error
+ return(list(
+   BB = batas_bawah,
+   BA = batas_atas
+ ))
+ }
+ }
+ }
+ # Input
+ p_hat <- 1.5
+ n <- 10
+ alpha <- 0.05
+
+ # Validasi
+ if (p_hat < 0 || p_hat > 1) {
+   cat("Error: p_hat harus berada antara 0 dan 1\n")
+ } else if (n <= 0) {
+   cat("Error: ukuran sampel harus lebih dari 0\n")
+ } else if (alpha != 0.05 && alpha != 0.10) {
+   cat("Error: alpha harus 0.05 atau 0.10\n")
+ } else {
+   hasil <- hitung_interval(p_hat, n, alpha)
+ }
+
+ #Menampilkan Hasil
+ cat("Batas bawah =", hasil$BB, "\n")
+ cat("Batas atas =", hasil$BA, "\n")
+ cat("Interval konfidensi = (", round(hasil$BB, 4), ", ", round(hasil$BA, 4), ")\n")
+ }
+ }
Error: p_hat harus berada antara 0 dan 1
>
```

Gambar 6.34 Input dan Output R Kondisi 2



```
colab.research.google.com/drive/1UGwlzV1NSCel-hvyNCauAaYU1oITLa#scrollTo=QwF6qILxauC
Untitled7.ipynb
File Edit Lihat Sisipkan Runtime Fitur Bantuan
Perintah + Kode + Teks ▶ Jalankan semua
[2] ✓ 0.0
# Input
p_hat = 1.5
n = 10
alpha = 0.05

# Validasi
if p_hat < 0 or p_hat > 1:
    print("Error: Proporsi sampel harus berada pada rentang 0 sampai 1")

elif n <= 0:
    print("Error: Ukuran sampel harus lebih dari 0")

elif alpha != 0.05 and alpha != 0.10:
    print("Error: Alpha harus bernilai 0.05 atau 0.10")

else:
    hasil = hitung_interval(p_hat, n, alpha)

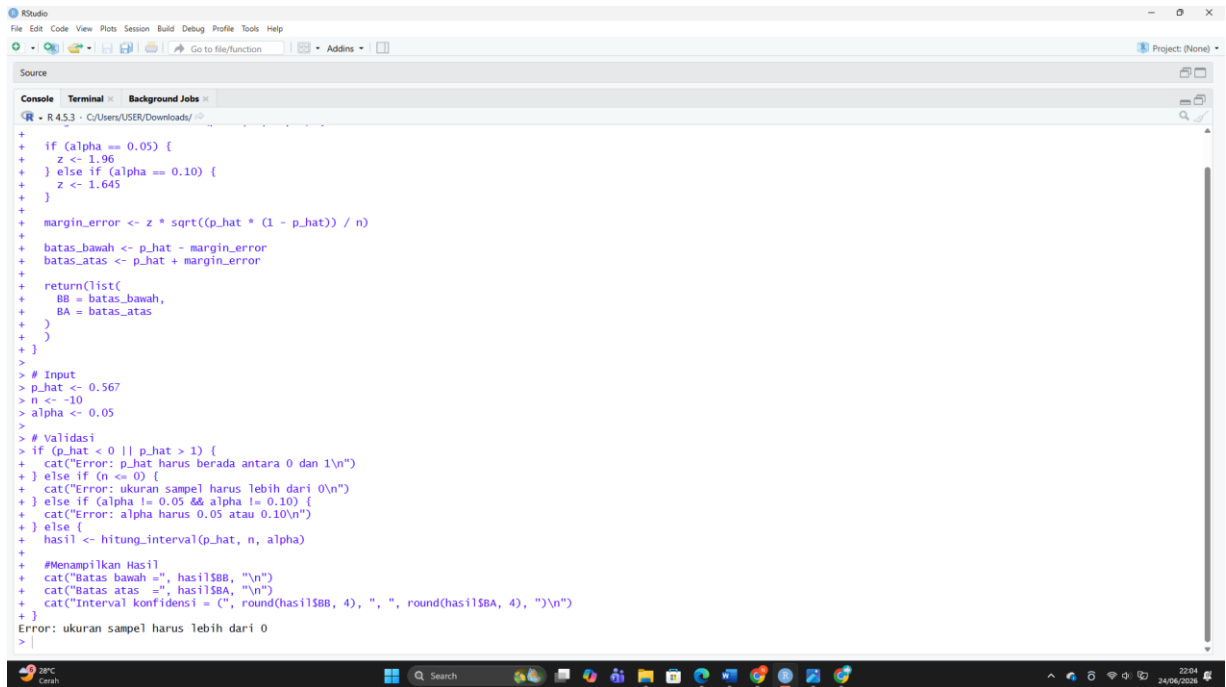
# Menampilkan Hasil
print("Batas Bawah =", hasil["BB"])
print("Batas Atas =", hasil["BA"])
print("Interval konfidensi = (", round(hasil["BB"], 4), ", ", round(hasil["BA"], 4), ")")

... Error: Proporsi sampel harus berada pada rentang 0 sampai 1
```

Gambar 6.35 Input dan Output Python Kondisi 2

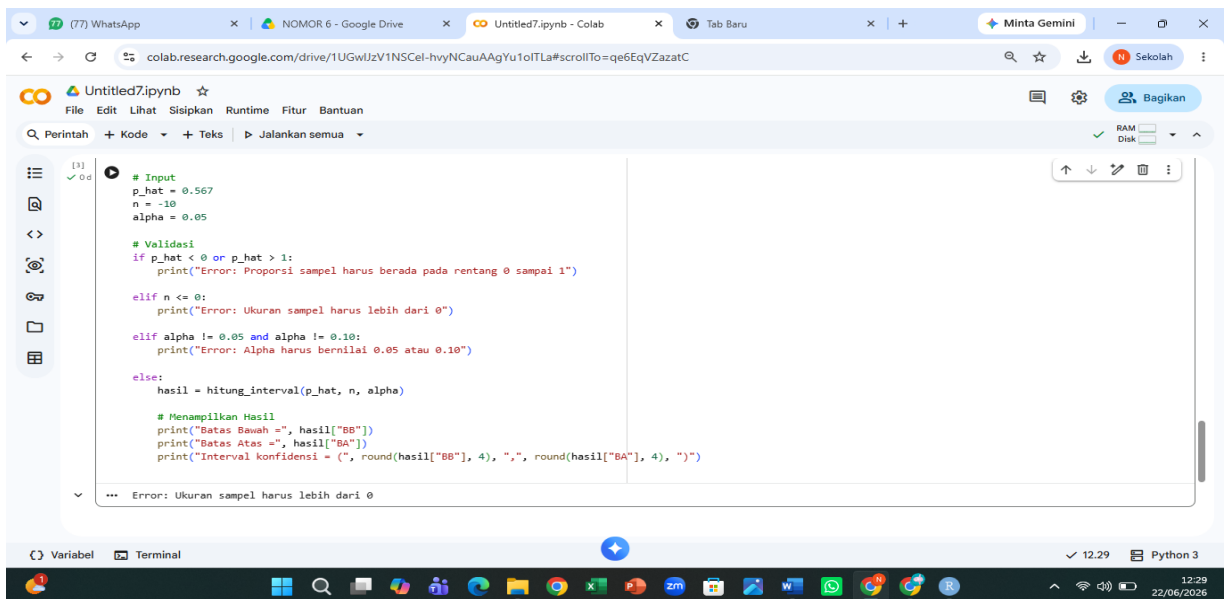
Interpretasi output :

Pada pengujian kedua, digunakan input $p_hat = 1,5$, $n = 10$, dan $\alpha = 0,05$. Program menampilkan pesan kesalahan karena nilai proporsi sampel berada di luar rentang yang diperbolehkan, yaitu antara 0 sampai 1. Hasil ini menunjukkan bahwa validasi terhadap nilai proporsi sampel telah berjalan dengan baik sehingga dapat mencegah terjadinya perhitungan menggunakan data yang tidak valid.



```
RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Source
Console Terminal Background Jobs
R 4.5.3 - C:/Users/USER/Downloads/
+ if (alpha == 0.05) {
+   z <- 1.96
+ } else if (alpha == 0.10) {
+   z <- 1.645
+ }
+ margin_error <- z * sqrt((p_hat * (1 - p_hat)) / n)
+ batas_bawah <- p_hat - margin_error
+ batas_atas <- p_hat + margin_error
+ return(list(
+   BB = batas_bawah,
+   BA = batas_atas
+ ))
+ }
+ }
> # Input
> p_hat <- 0.567
> n <- -10
> alpha <- 0.05
> # Validasi
> if (p_hat < 0 || p_hat > 1) {
+   cat("Error: p_hat harus berada antara 0 dan 1\n")
+ } else if (n <= 0) {
+   cat("Error: ukuran sampel harus lebih dari 0\n")
+ } else if (alpha != 0.05 && alpha != 0.10) {
+   cat("Error: alpha harus 0.05 atau 0.10\n")
+ } else {
+   hasil <- hitung_interval(p_hat, n, alpha)
+ }
+ #Menampilkan Hasil
+ cat("Batas bawah =", hasil$BB, "\n")
+ cat("Batas atas =", hasil$BA, "\n")
+ cat("Interval konfidensi = (", round(hasil$BB, 4), ", ", round(hasil$BA, 4), ")\n")
+ }
Error: ukuran sampel harus lebih dari 0
>
```

Gambar 6.36 Input dan Output R Kondisi 3



```
Untitled7.ipynb
File Edit Lihat Sisipkan Runtime Fitur Bantuan
Perintah + Kode + Teks ▶ Jalankan semua
# Input
p_hat = 0.567
n = -10
alpha = 0.05

# Validasi
if p_hat < 0 or p_hat > 1:
    print("Error: Proporsi sampel harus berada pada rentang 0 sampai 1")

elif n <= 0:
    print("Error: Ukuran sampel harus lebih dari 0")

elif alpha != 0.05 and alpha != 0.10:
    print("Error: Alpha harus bernilai 0.05 atau 0.10")

else:
    hasil = hitung_interval(p_hat, n, alpha)

# Menampilkan Hasil
print("Batas Bawah =", hasil["BB"])
print("Batas Atas =", hasil["BA"])
print("Interval konfidensi = (", round(hasil["BB"], 4), ", ", round(hasil["BA"], 4), ")")

... Error: Ukuran sampel harus lebih dari 0
```

Gambar 6.37 Input dan Output Python Kondisi 3

Pada pengujian ketiga, digunakan input $p_hat = 0,567$, $n = -10$, dan $\alpha = 0,05$. Program menampilkan pesan kesalahan karena ukuran sampel bernilai negatif. Ukuran sampel harus bernilai lebih dari nol agar perhitungan interval konfidensi dapat dilakukan. Hasil ini menunjukkan bahwa validasi terhadap ukuran sampel telah berjalan sesuai dengan ketentuan yang ditetapkan.

Gambar 6.39 Input dan Output R Kondisi 4

Gambar 6.39 Input dan Output Python Kondisi 4

Interpretasi output :

Pada pengujian keempat, digunakan input $p_hat = 0,567$, $n = 10$, dan $\alpha = 0,01$. Program menampilkan pesan kesalahan karena nilai α yang dimasukkan tidak termasuk nilai yang didukung oleh program, yaitu 0,05 dan 0,10. Hasil ini menunjukkan bahwa validasi terhadap tingkat signifikansi telah berjalan dengan baik sehingga hanya nilai α yang sesuai yang dapat digunakan dalam perhitungan.

BAB VII

PENUTUP

7.1 Kesimpulan

Berdasarkan pengerjaan program yang sudah dilakukan sebelumnya dapat diketahui bahwa sebuah algoritma pemrograman dapat sangat membantu berbagai persoalan didalam kehidupan, dengan pemanfaatan algoritma pemrograman, suatu proses yang sebelumnya membutuhkan waktu dan tenaga yang banyak menjadi lebih efisien. Dapat dilihat juga bahwa algoritma akan menampilkan hasil yang tepat ketika penulisan intruksi juga jelas sehingga komputer dapat memahami perintah untuk melakukan eksekusi. Karena kita menggunakan dua buah perangkat lunak pada project ini, kita dapat melihat bahwa meskipun Bahasa R dan Python memiliki perbedaan sintaks, keduanya mampu menghasilkan logika program dan *output* yang setara dan valid untuk kasus yang sama. Oleh sebab itu, dapat disimpulkan bahwa pemahaman mengenai logika yang berjalan pada algoritma sangat penting dipahami dan mempelajari bagaimana cara mengembangkan program tersebut agar dapat melakukan proses yang lebih kompleks.

7.2 Kendala

Selama mengerjakan project ini, kendala yang dihadapi oleh kami adalah bagaimana membentuk program agar menghasilkan keluaran yang rapih dan hasilnya lebih mudah dimengerti oleh penerima. Riset mendalam mengenai batasan masalah setiap program tersebut juga memerlukan ketelitian karena dengan mengetahui Batasan program kami dapat mengetahui apa yang tidak bisa dijalankan oleh program dan membuat terobosan atau melakukan pengembangan program agar dapat menyelesaikan permasalahan yang sebelumnya menjadi batasan masalah.

7.3 Saran

Berdasarkan pengerjaan program yang sudah dilakukan, terdapat beberapa saran yang dapat digunakan untuk mengembangkan program sebagai berikut :

1. Penanganan error yang lebih kompleks

Proses validasi input dapat ditingkatkan bukan hanya dalam lingkup tipe data angka saja ada sebuah program yang melibatkan perhitungan matematis tetapi dapat menggunakan blok lain di Python atau R untuk mencegah crash misalnya pengguna

tidak sengaja memasukkan karakter huruf.

2. Pembersihan teks otomatis

Pada program yang melibatkan perhitungan kata, program dapat dikembangkan agar dapat secara otomatis menghapus tanda baca lain sebelum menghitung kata. Ini penting karena Ketika kata yang menempel dengan tanda baca itu tidak terhitung sebagai kata yang berbeda.

LAMPIRAN

Repository Github

<https://github.com/Luqyana79/PROJEK-ALPRO-KELOMPOK-B>

Alat Bantu AI

Soal 1: <https://chatgpt.com/share/6a2a42fa-d7d4-83ec-adbe-73cf5c1b858b>

Soal 2: <https://chatgpt.com/share/6a2e9540-2c3c-83ec-a31a-8c03935dd1f6>

Soal 3: <https://gemini.google.com/share/d/1w7wyeddxbut0JxsaQdEBjG-l3SYJP4FN>

Soal 4: <https://claude.ai/share/11aa5d32-f9ec-44ce-bc46-3bd60b109a08>

Soal 5: <https://chatgpt.com/share/6a2b1011-028c-83ec-914f-7718e6b0b962>

Soal 6: <https://chatgpt.com/share/6a26cf8b-836c-83ec-a5a6-dbe040baff5b>

Link Video Demo

https://drive.google.com/file/d/1jeCq8M_JMrNWKIEd-MD6_0jSuugXRdDO/view